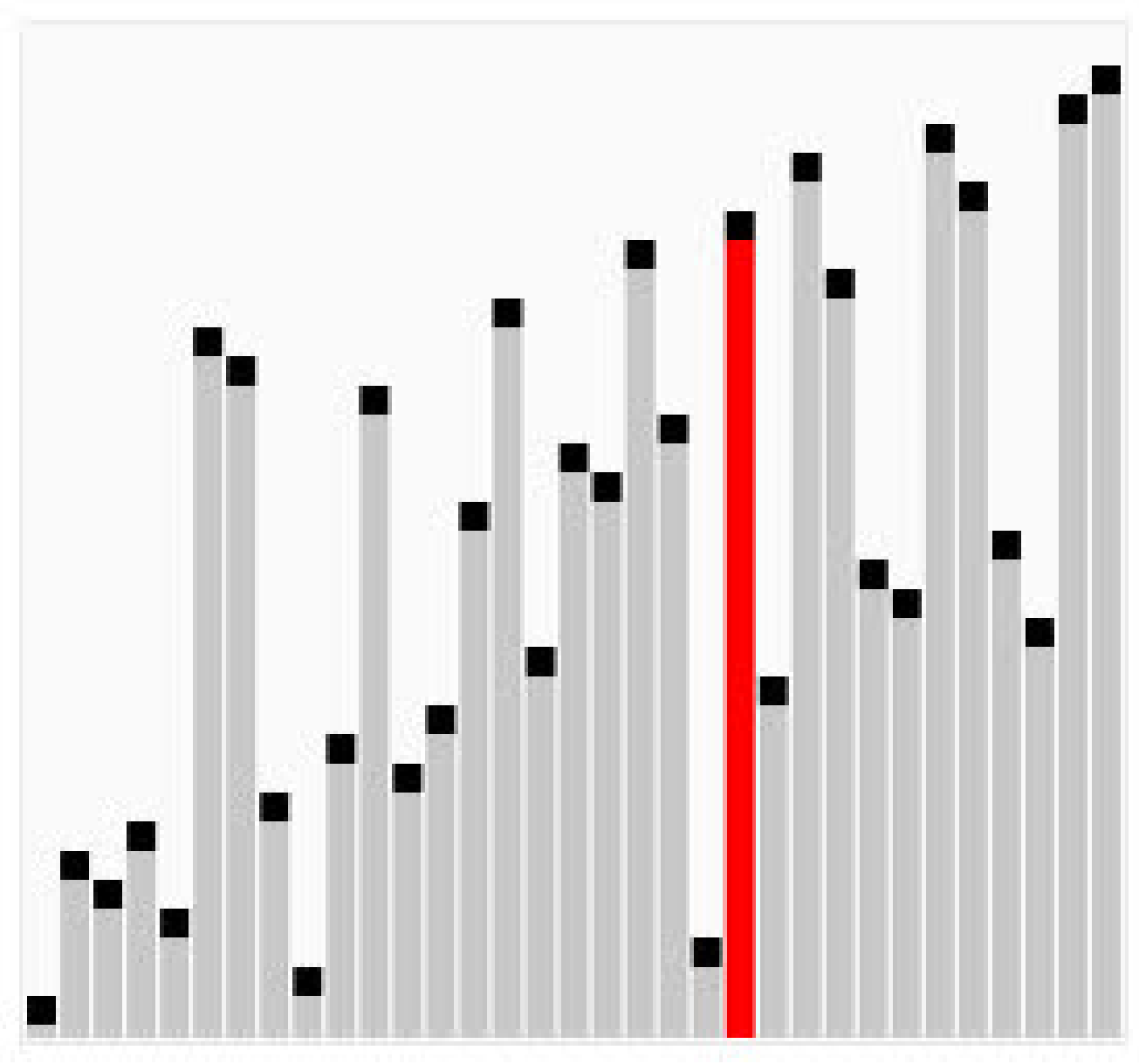


SORT ALGORITAM

(PROJEKTNA IMPLEMENTACIJA)

Predmet: Paralelni algoritmi

PRIRODNO - MATEMATIČKI FAKULTET
RAČUNARSTVO I INFORMACIONE TEHNOLOGIJE



BUBBLE SORT

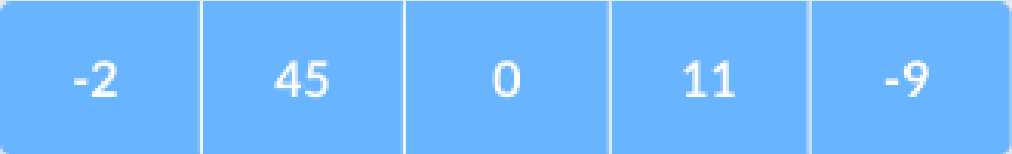
- Poredi i zamjenjuje susjedne elemente ako nisu u pravom redoslijedu.
- Najveći (ili najmanji) element „ispliva” na kraj nakon svake iteracije.
- Sa svakom iteracijom pretraga se skraćuje za jedan element.
- Jednostavan za implementaciju, neefikasan za velike skupove podataka.
- Vremenska složenost: $O(n^2)$



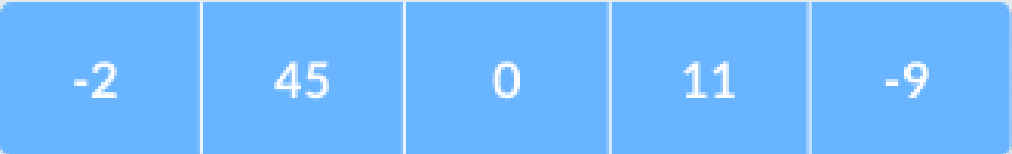


step = 0

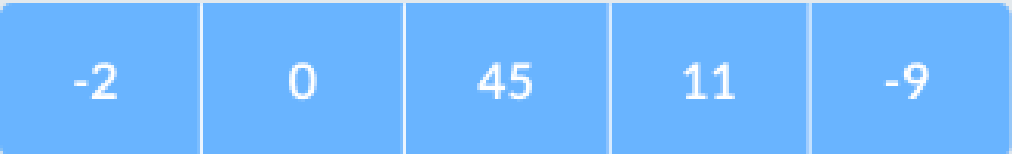
i = 0



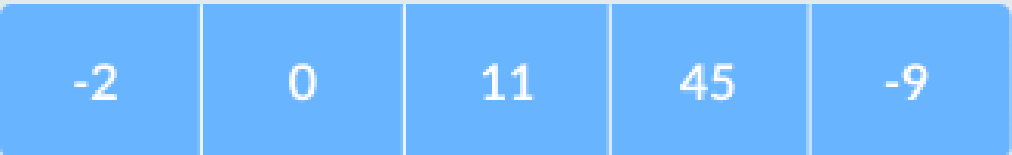
i = 1



i = 2

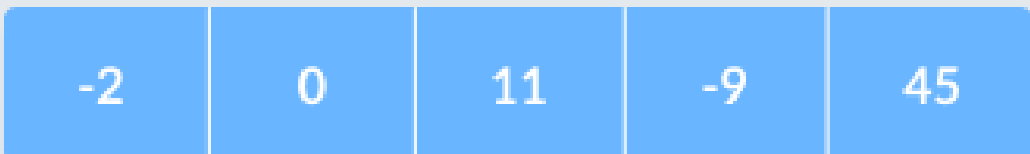


i = 3

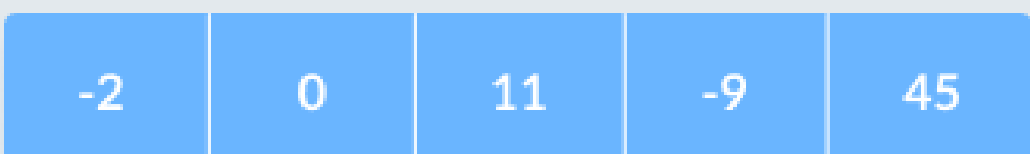


step = 1

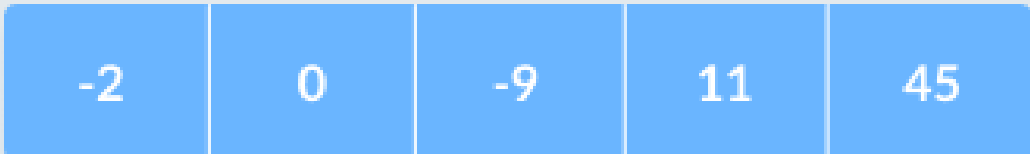
i = 0



i = 1



i = 2

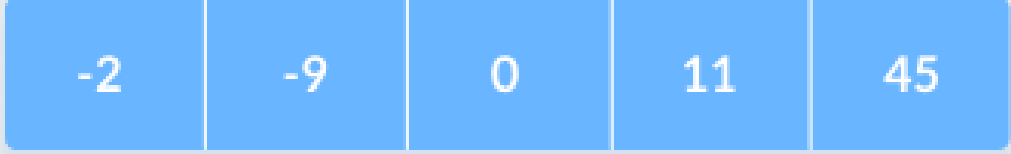
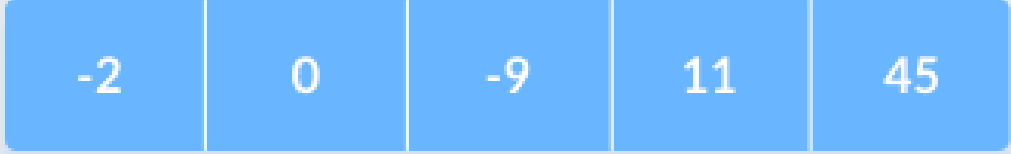


step = 2

i = 0

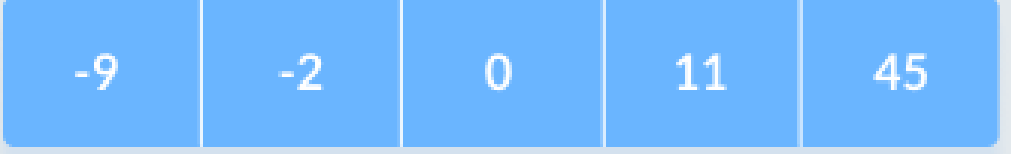
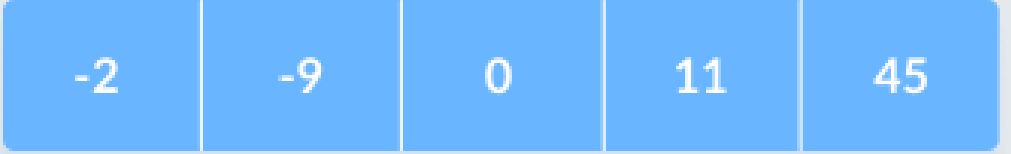


i = 1



step = 3

i = 0



Sekvencijalni algoritam



```
void bubbleSort(int arr[], int n) {  
    bool sorted = false;  
    int tmp;  
  
    while (!sorted) {  
        sorted = true;  
  
        for (int i = 0; i < n-1; ++i) {  
            if (arr[i] > arr[i+1]) {  
                tmp = arr[i];  
                arr[i] = arr[i+1];  
                arr[i+1] = tmp;  
                sorted = false;  
            }  
        }  
    }  
}
```

Paralelni algoritam



```
void bubbleSortParallel(int arr[], int n) {
    bool sorted = false;
    int tmp;

    while (!sorted) {
        sorted = true;

        #pragma omp parallel private(tmp)
        {
            // Even phase
            #pragma omp for reduction(&&:sorted)
            for (int i = 0; i < n-1; i += 2) {
                if (arr[i] > arr[i+1]) {
                    tmp = arr[i];
                    arr[i] = arr[i+1];
                    arr[i+1] = tmp;
                    sorted = false;
                }
            }

            // Odd phase
            #pragma omp for reduction(&&:sorted)
            for (int i = 1; i < n-1; i += 2) {
                if (arr[i] > arr[i+1]) {
                    tmp = arr[i];
                    arr[i] = arr[i+1];
                    arr[i+1] = tmp;
                    sorted = false;
                }
            }
        }
    }
}
```


ANALIZA REZULTATA

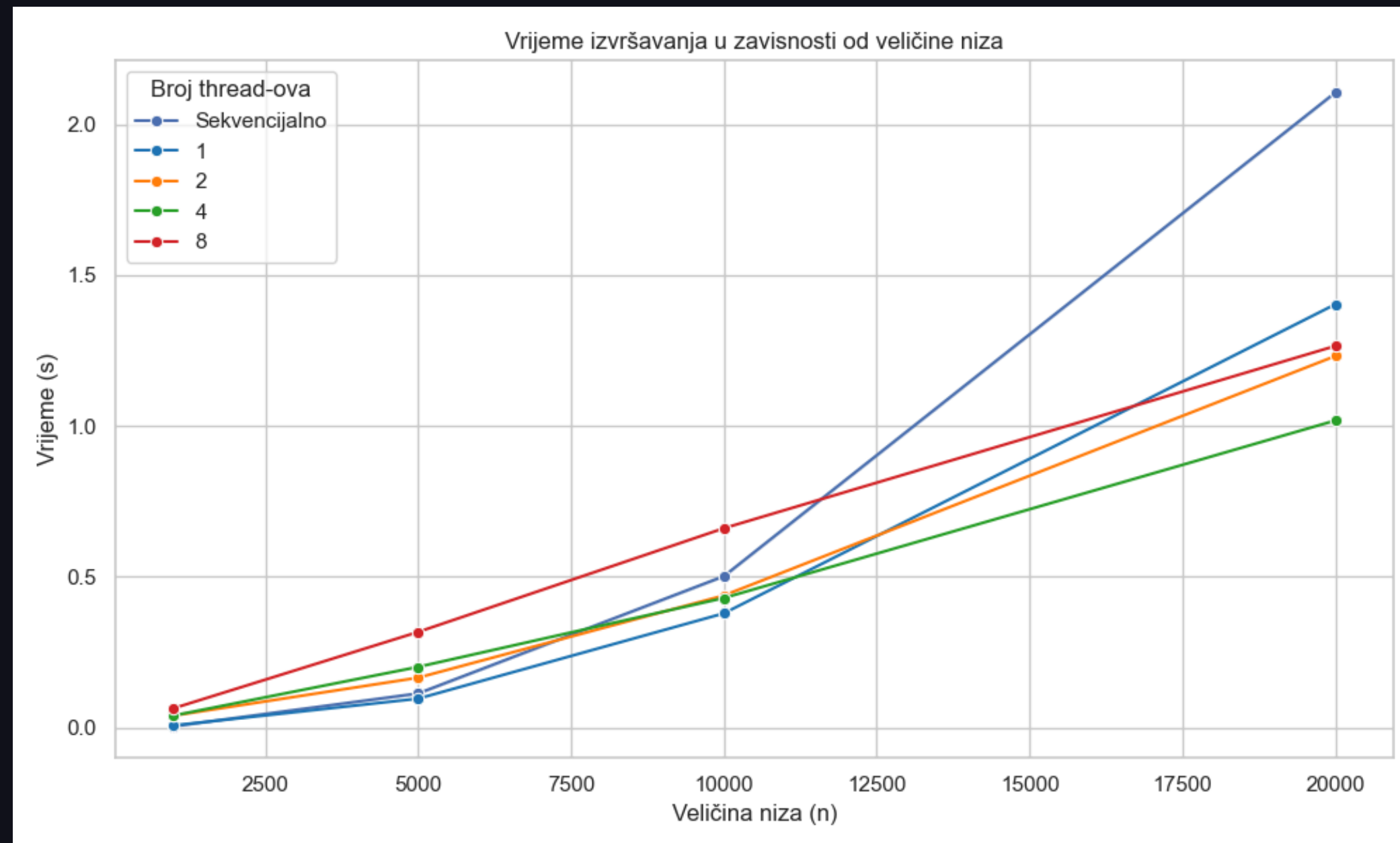


- Paralelizacija nije efikasna za male ulaze (nizovi veličine 1000 i 5000)
- Prekomjerno paralelizovanje može biti kontraproduktivno (najniži speedup (0.0711) i efikasnost (0.0089) zabilježeni su pri 8 niti i n=1000)
- Za veće ulaze (n = 20000), paralelizacija donosi dobitak (maksimalni speedup od 2.068 postignut je sa 4 niti)
- Više niti ne znači nužno bolje performanse (raste ubrzanje ali i trošak (10.12 za 8 niti), a efikasnost opada)
- Zaključak: Bubble Sort ima ograničenu skalabilnost

n	threads	seq_time	par_time	speedup	efficiency	cost
1000	1	0.004551	0.0072	0.632139	0.632139	0.0072
1000	2	0.004551	0.0398	0.114357	0.057178	0.0796
1000	4	0.004551	0.04	0.113785	0.028446	0.16
1000	8	0.004551	0.064	0.071116	0.008889	0.512
5000	1	0.113279	0.096	1.179994	1	0.096
5000	2	0.113279	0.1654	0.684882	0.342441	0.3308
5000	4	0.113279	0.2012	0.563019	0.140755	0.8048
5000	8	0.113279	0.317	0.357348	0.044669	2.536
10000	1	0.501972	0.3788	1.325164	1	0.3788
10000	2	0.501972	0.438	1.146055	0.573028	0.876
10000	4	0.501972	0.4296	1.168464	0.292116	1.7184
10000	8	0.501972	0.6608	0.759643	0.094955	5.2864
20000	1	2.105273	1.4032	1.500337	1	1.4032
20000	2	2.105273	1.2316	1.70938	0.85469	2.4632
20000	4	2.105273	1.018	2.068048	0.517012	4.072
20000	8	2.105273	1.265	1.664247	0.208031	10.12

ANALIZA REZULTATA

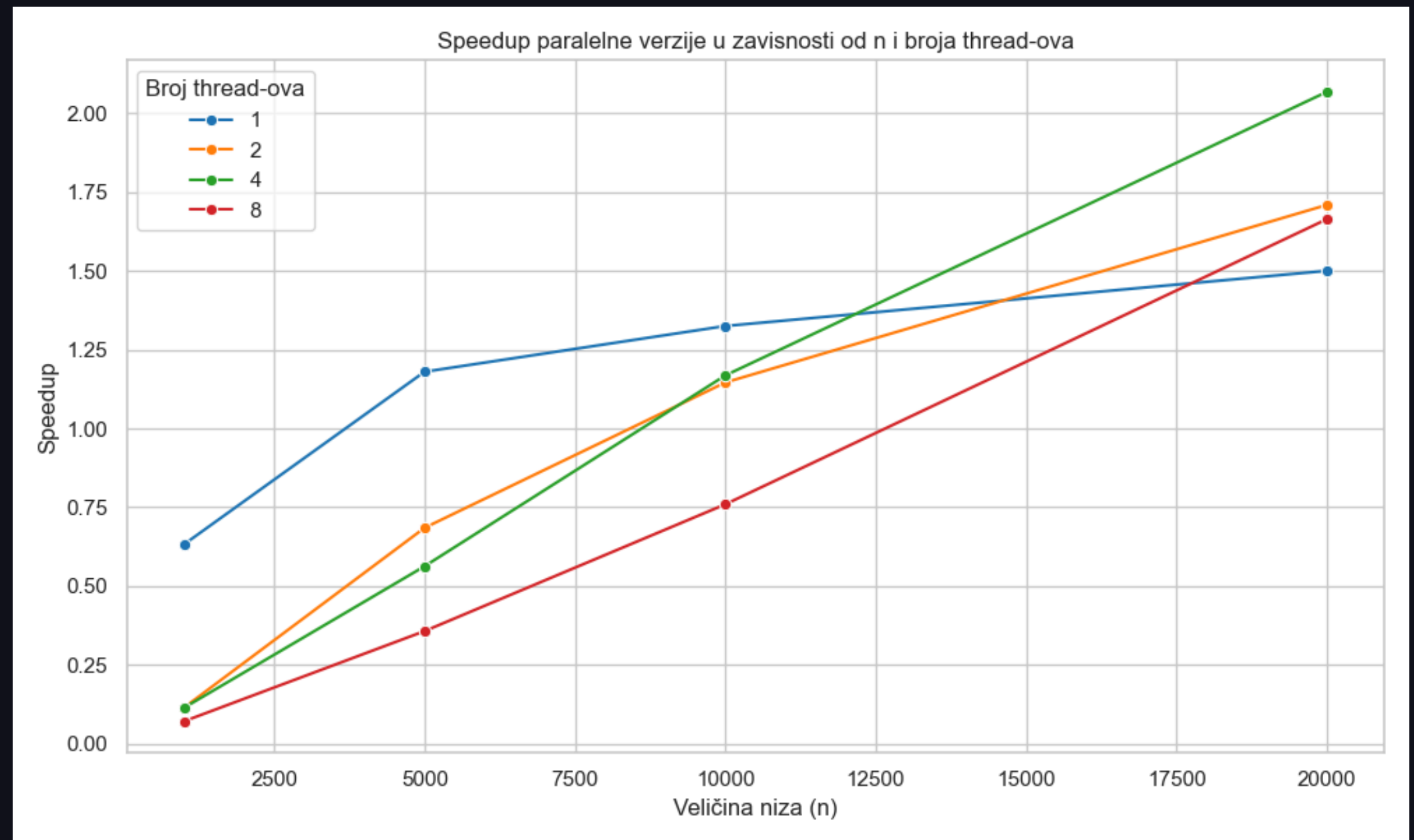
- Vrijeme izvršavanja raste linearno sa veličinom niza.
- Sekvencijalna verzija najbrža za ulaze do 5000 elemenata.
- 4 niti najefikasnije za velike ulaze ($n = 20000$).
- 8 niti usporava zbog dodatne režije i sinhronizacije.



ANALIZA REZULTATA

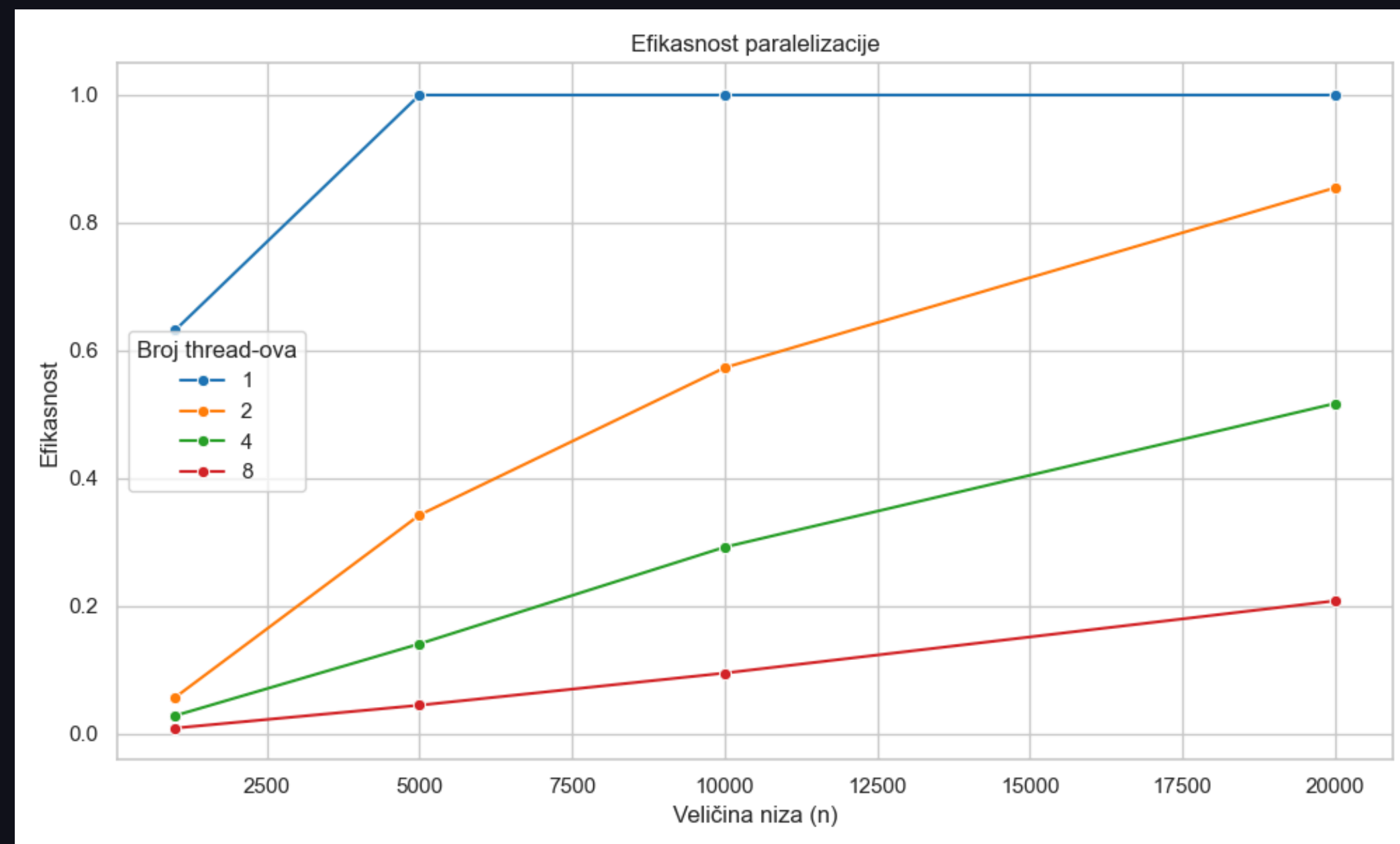


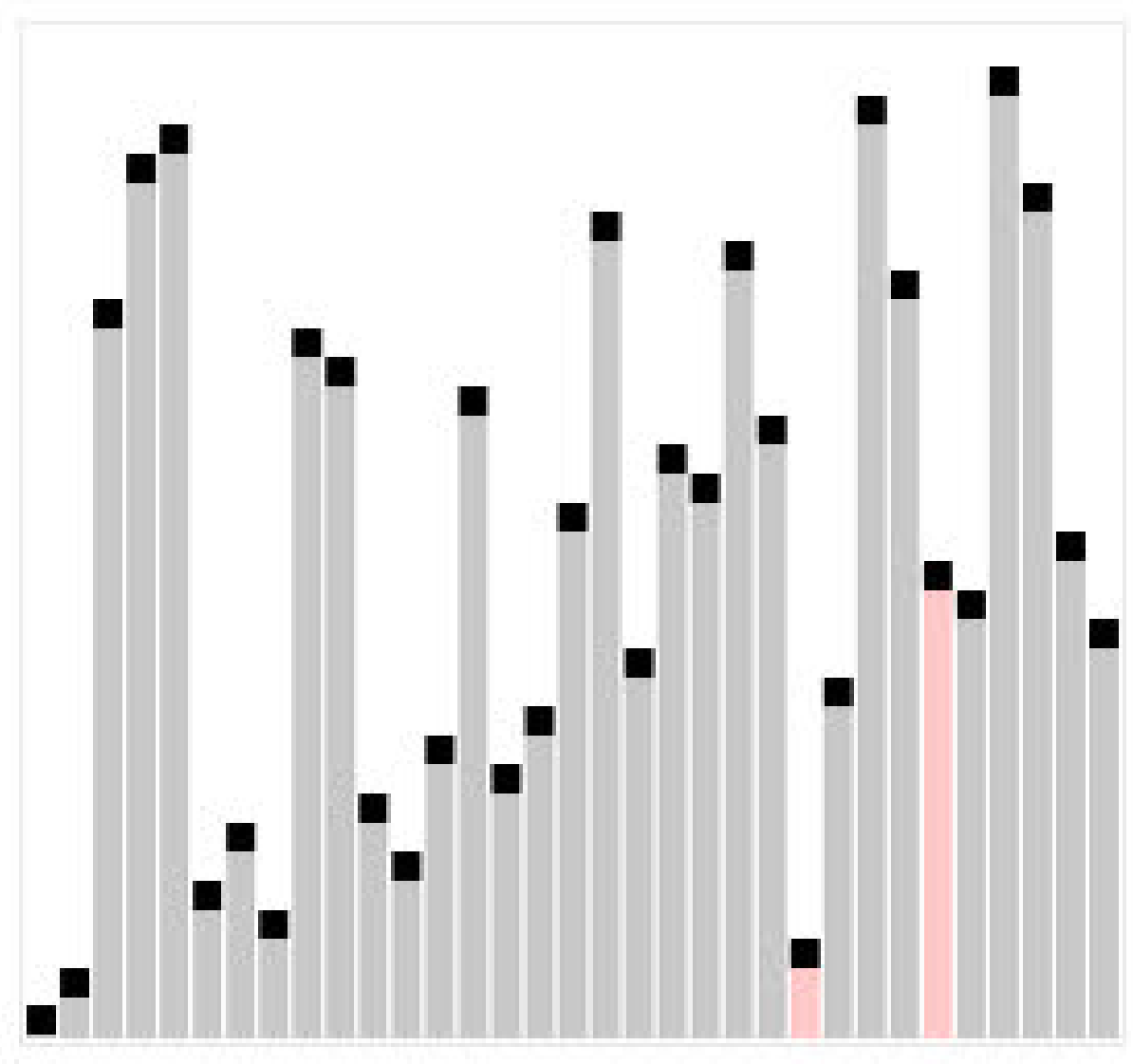
- Speedup raste sa veličinom niza i brojem niti, ali ne proporcionalno.
- Najveći speedup (~2.07) postignut je za $n = 20000$ i 4 niti.
- 8 niti ne donosi dodatno ubrzanje zbog režije paralelizacije.
- Više niti $\neq$ nužno bolje performanse.



ANALIZA REZULTATA

- Efikasnost opada sa povećanjem broja niti.
- Veći ulazi poboljšavaju efikasnost.
- 1 nit = idealna efikasnost (1.0).
- 8 niti = veoma niska efikasnost za male ulaze.
- Paralelizacija ima smisla samo za velike nizove i umjeren broj niti.





SELECTION SORT

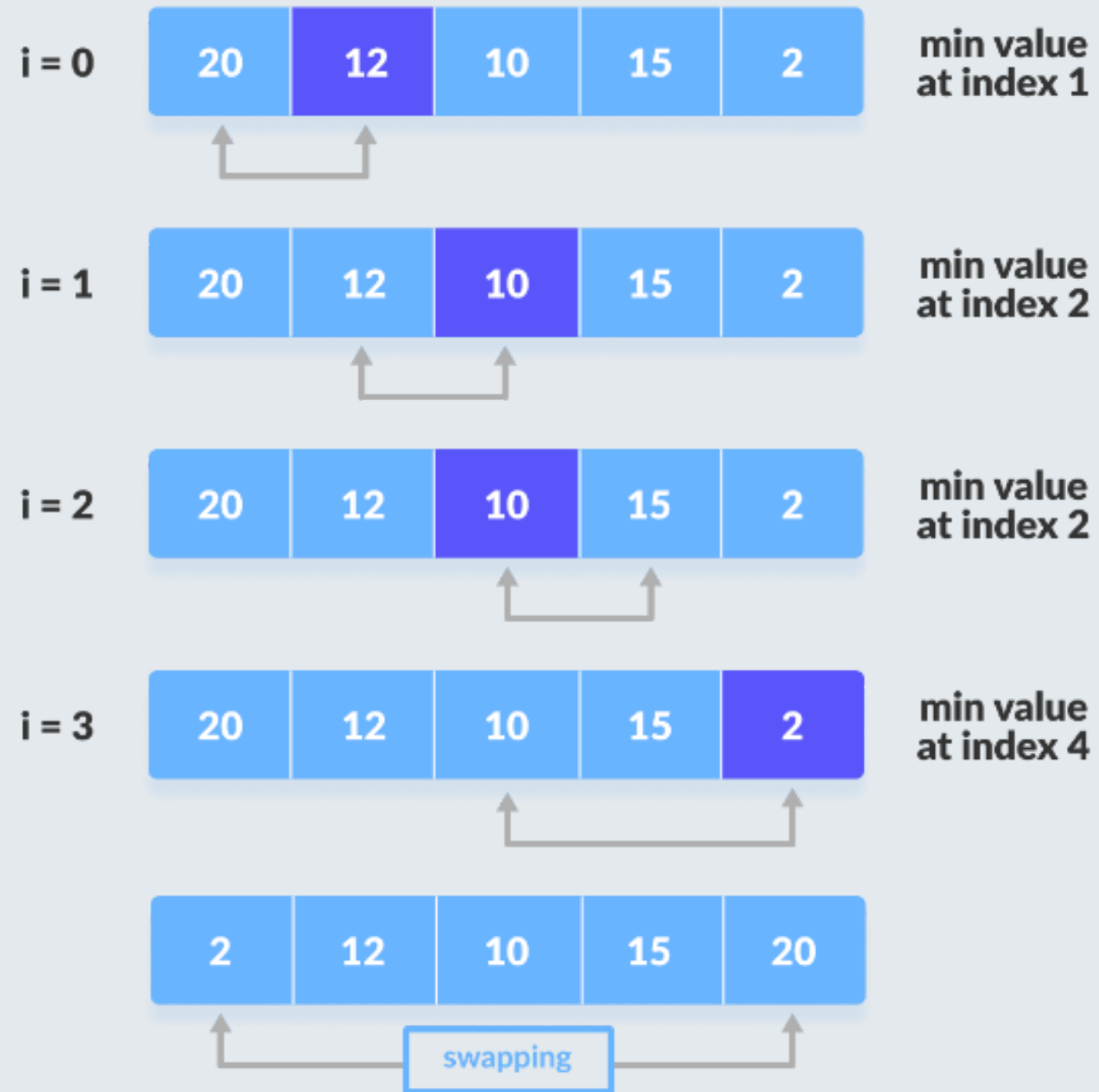


- Pronalazi se najmanji ili najveći element iz nesortiranog dijela i smješta se na početak tog dijela.
- Nakon svake iteracije, sortirani dio raste, a nesortirani se smanjuje.
- Jednostavna implementacija, ali neefikasan za velike podatke.
- Vremenska složenost: $O(n^2)$

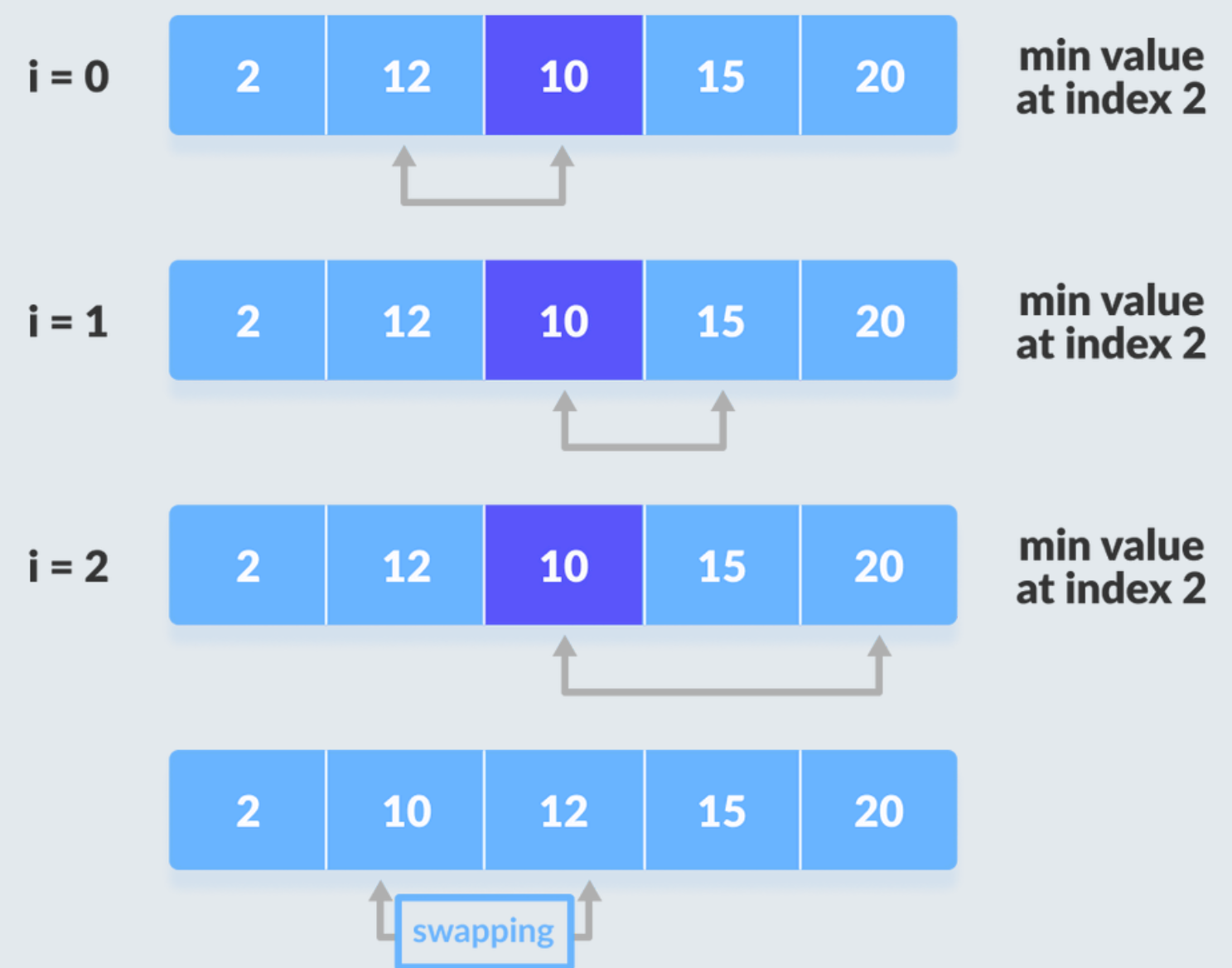




step = 0



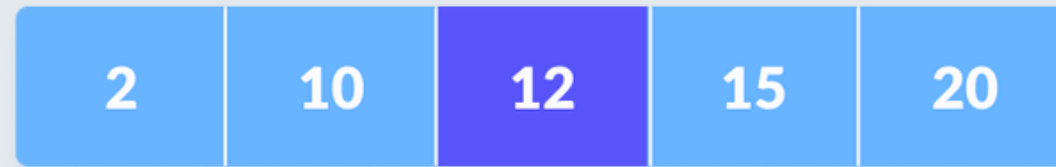
step = 1



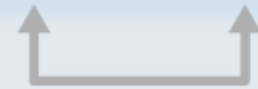


step = 2

i = 0



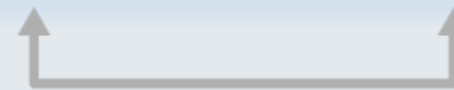
min value
at index 2



i = 2



min value
at index 2



already in place

step = 3

i = 0



min value
at index 3



already in place

Sekvencijalni algoritam



```
#include <algorithm>

void selectionSort(int* arr, int n) {
    for (int i = 0; i < n - 1; ++i) {
        int min_idx = i;
        for (int j = i + 1; j < n; ++j)
            if (arr[j] < arr[min_idx])
                min_idx = j;
        std::swap(arr[i], arr[min_idx]);
    }
}
```


Paralelni algoritam



```
#include <omp.h>
#include <algorithm>

void selectionSortParallel(int* arr, int n) {
    for (int i = 0; i < n - 1; ++i) {
        int min_idx = i;

        #pragma omp parallel
        {
            int local_min_idx = min_idx;

            #pragma omp for nowait
            for (int j = i + 1; j < n; ++j) {
                if (arr[j] < arr[local_min_idx]) {
                    local_min_idx = j;
                }
            }

            #pragma omp critical
            {
                if (arr[local_min_idx] < arr[min_idx]) {
                    min_idx = local_min_idx;
                }
            }
        }

        std::swap(arr[i], arr[min_idx]);
    }
}
```

ANALIZA REZULTATA

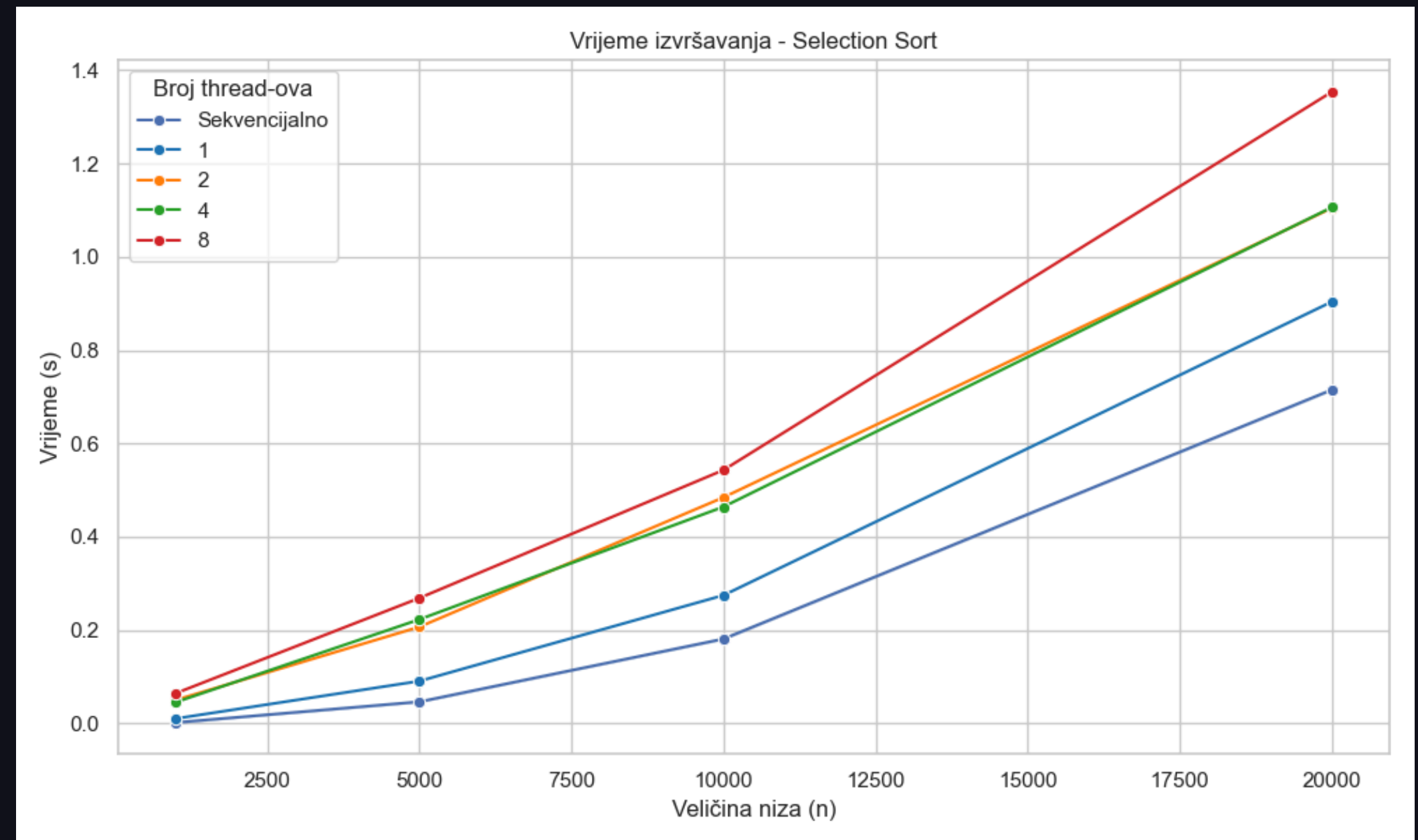
- Paralelizacija ne poboljšava performanse – u većini slučajeva dolazi do usporavanja.
- Najveći speedup = 0.79 (n = 20000, 2 niti).
- Najniži speedup = 0.028 (n = 1000, 8 niti)
→ prekomjerna paralelizacija je kontraproduktivna.
- Efikasnost opada sa više niti, kreće se od 0.32 do ispod 0.01.
- Trošak (cost) drastično raste sa više niti – najveći zabilježeni je 10.83 (n = 20000, 8 niti).
- Zaključak: Selection Sort nije pogodan za paralelizaciju zbog izražene sekvencijalnosti koraka.

n	threads	seq_time	par_time	speedup	efficiency	cost
1000	1	0.001864	0.0104	0.179269	0.179269	0.0104
1000	2	0.001864	0.0504	0.036992	0.018496	0.1008
1000	4	0.001864	0.0456	0.040886	0.010221	0.1824
1000	8	0.001864	0.0648	0.028772	0.003596	0.5184
5000	1	0.046315	0.0906	0.511199	0.511199	0.0906
5000	2	0.046315	0.2068	0.223958	0.111979	0.4136
5000	4	0.046315	0.2228	0.207875	0.051969	0.8912
5000	8	0.046315	0.2686	0.17243	0.021554	2.1488
10000	1	0.181046	0.2746	0.65931	0.65931	0.2746
10000	2	0.181046	0.4844	0.373754	0.186877	0.9688
10000	4	0.181046	0.4642	0.390018	0.097505	1.8568
10000	8	0.181046	0.543	0.333419	0.041677	4.344
20000	1	0.714962	0.9036	0.791238	0.791238	0.9036
20000	2	0.714962	1.1044	0.647376	0.323688	2.2088
20000	4	0.714962	1.106	0.64644	0.16161	4.424
20000	8	0.714962	1.3534	0.528271	0.066034	10.8272

ANALIZA REZULTATA



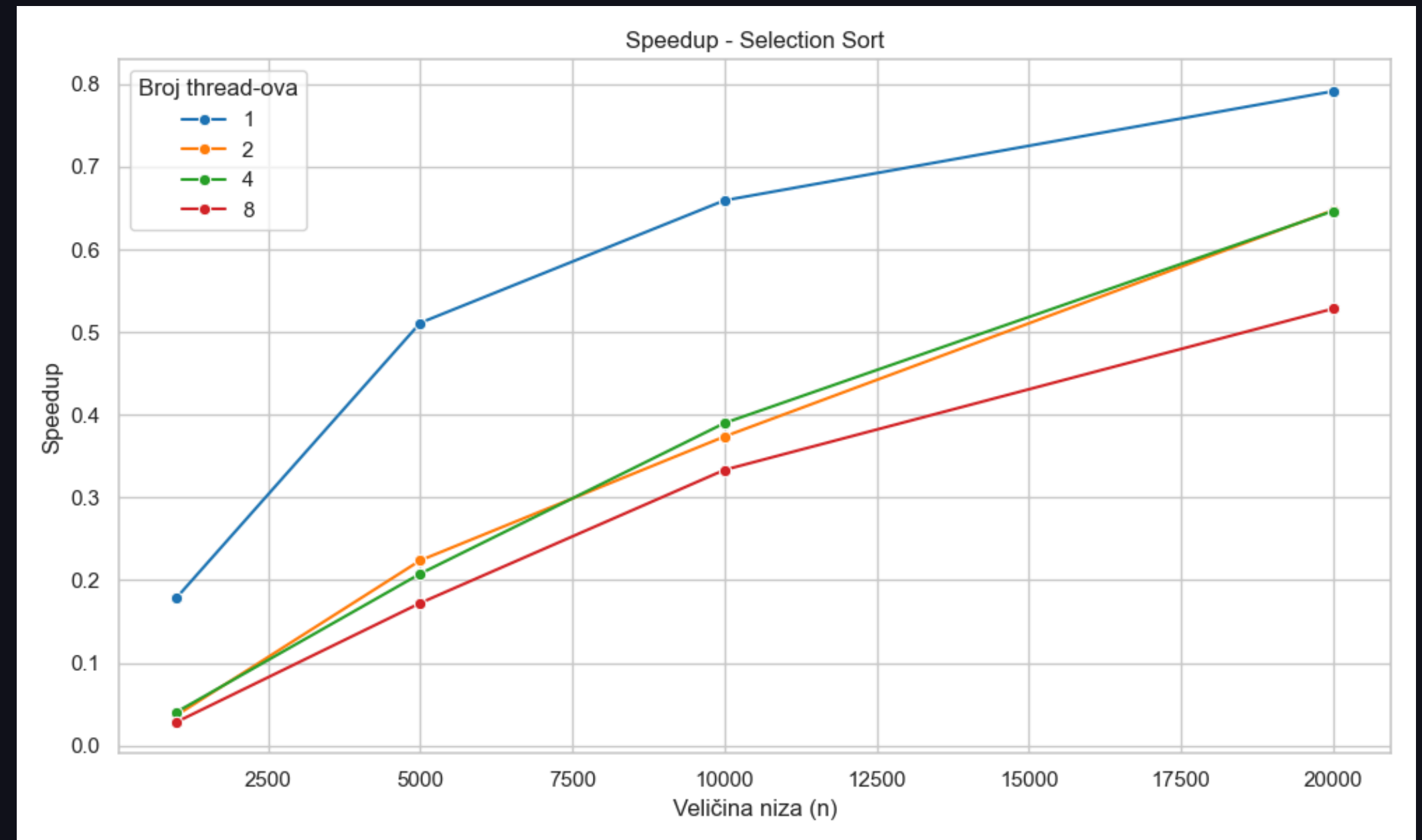
- Paralelne verzije sa 4 i 8 niti sporije su od sekvencijalne.
- Vrijeme izvršavanja raste sa veličinom ulaza kod većeg broja niti.
- Overhead paralelizacije nadmašuje dobit od raspodjele posla.
- Paralelizacija Selection Sort-a donosi više štete nego koristi.



ANALIZA REZULTATA



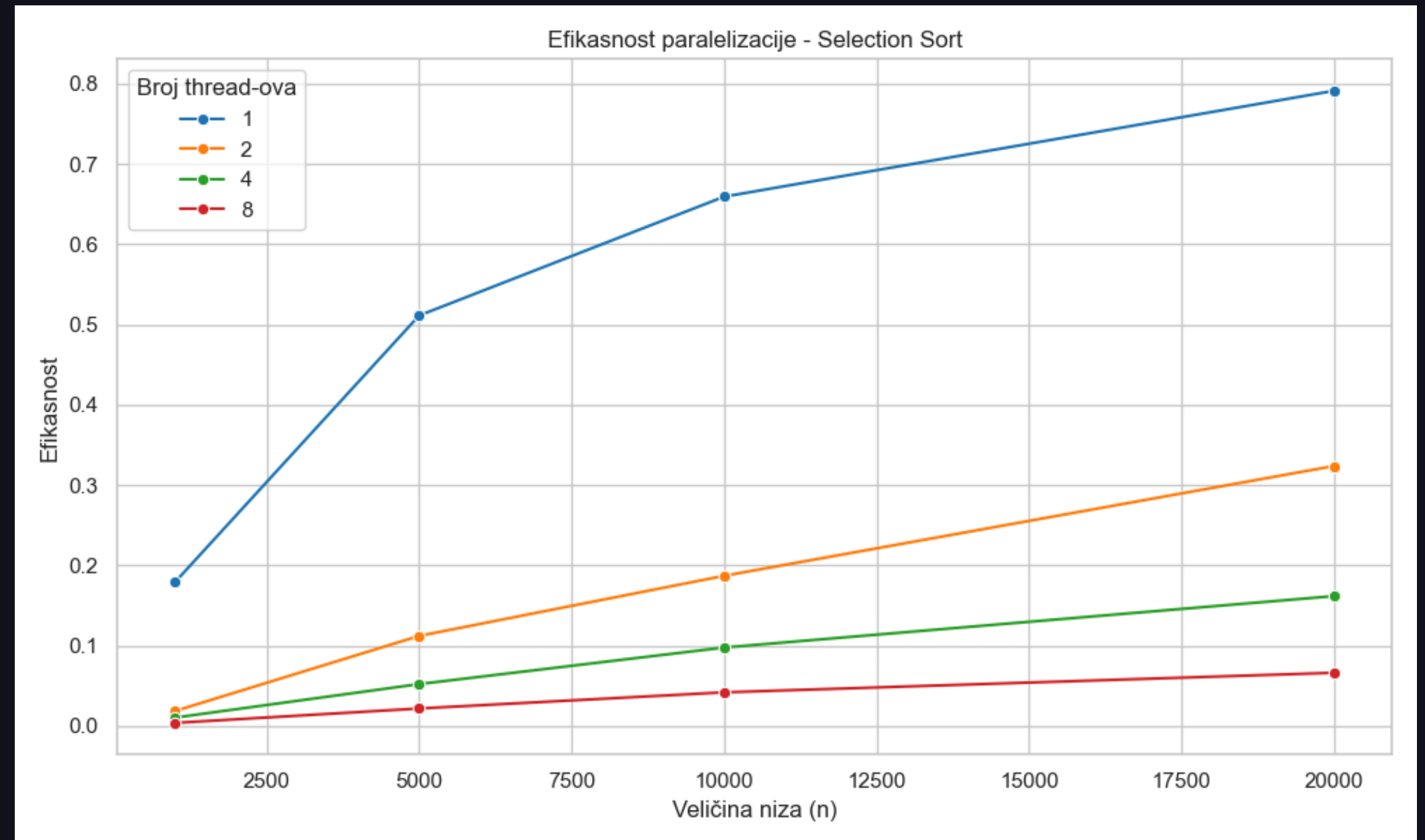
- Speedup ostaje ispod 0.8 čak i za $n = 20000$.
- Najbolji rezultat daje 2 niti, dok 4 i 8 niti pogoršavaju performanse.
- Povećanje broja niti ne donosi koristi.
- Rezultati potvrđuju teorijsko ograničenje paralelizacije kod zavisnih algoritama.

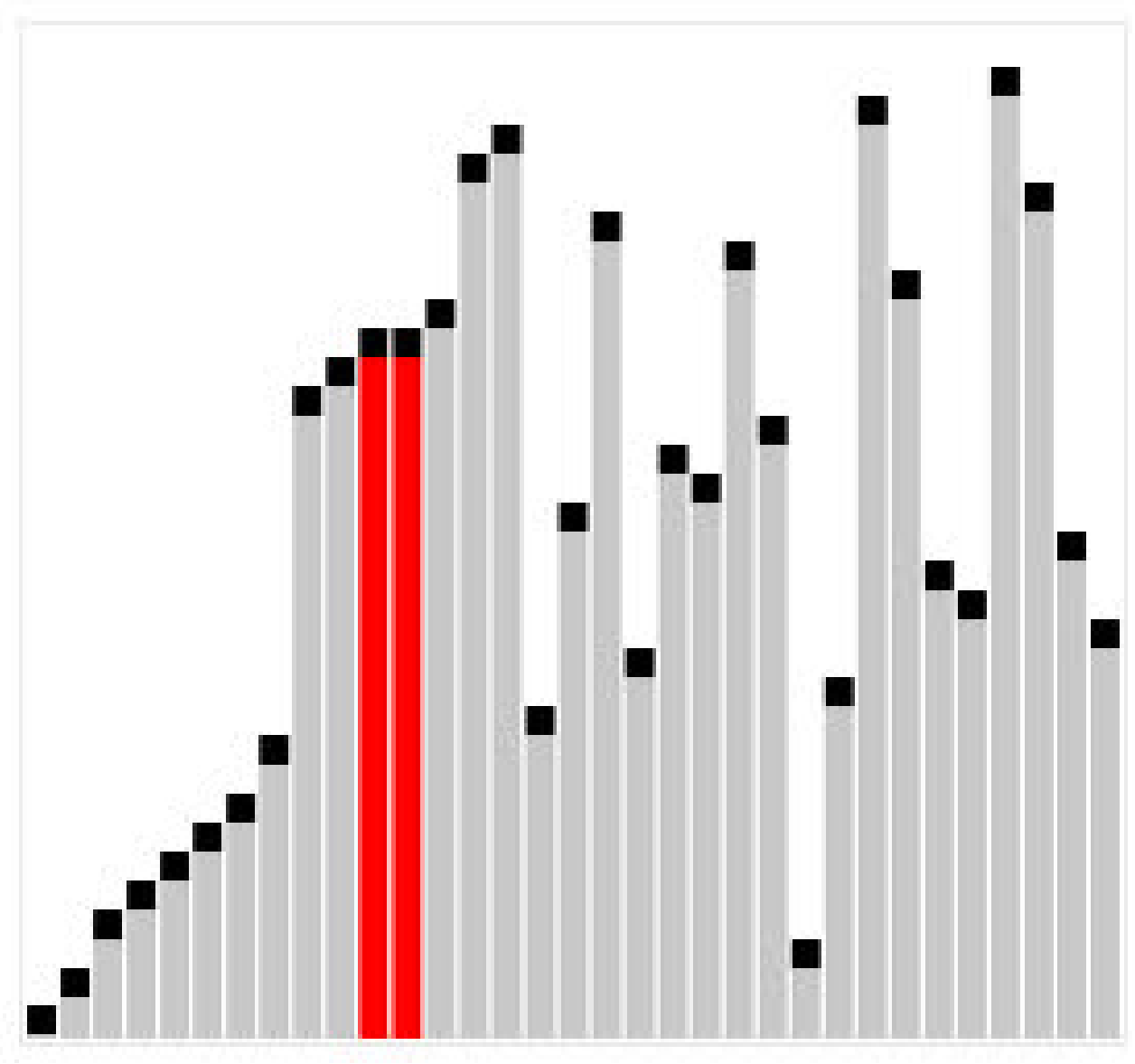


ANALIZA REZULTATA



- Efikasnost drastično opada sa povećanjem broja niti.
- Kod 8 niti efikasnost je ispod 0.1 za sve veličine ulaza.
- Paralelizacija Selection Sort-a dovodi do degradacije performansi.
- Algoritam nije prikladan za paralelno izvršavanje zbog svoje sekvencijalne prirode.





INSERTION SORT

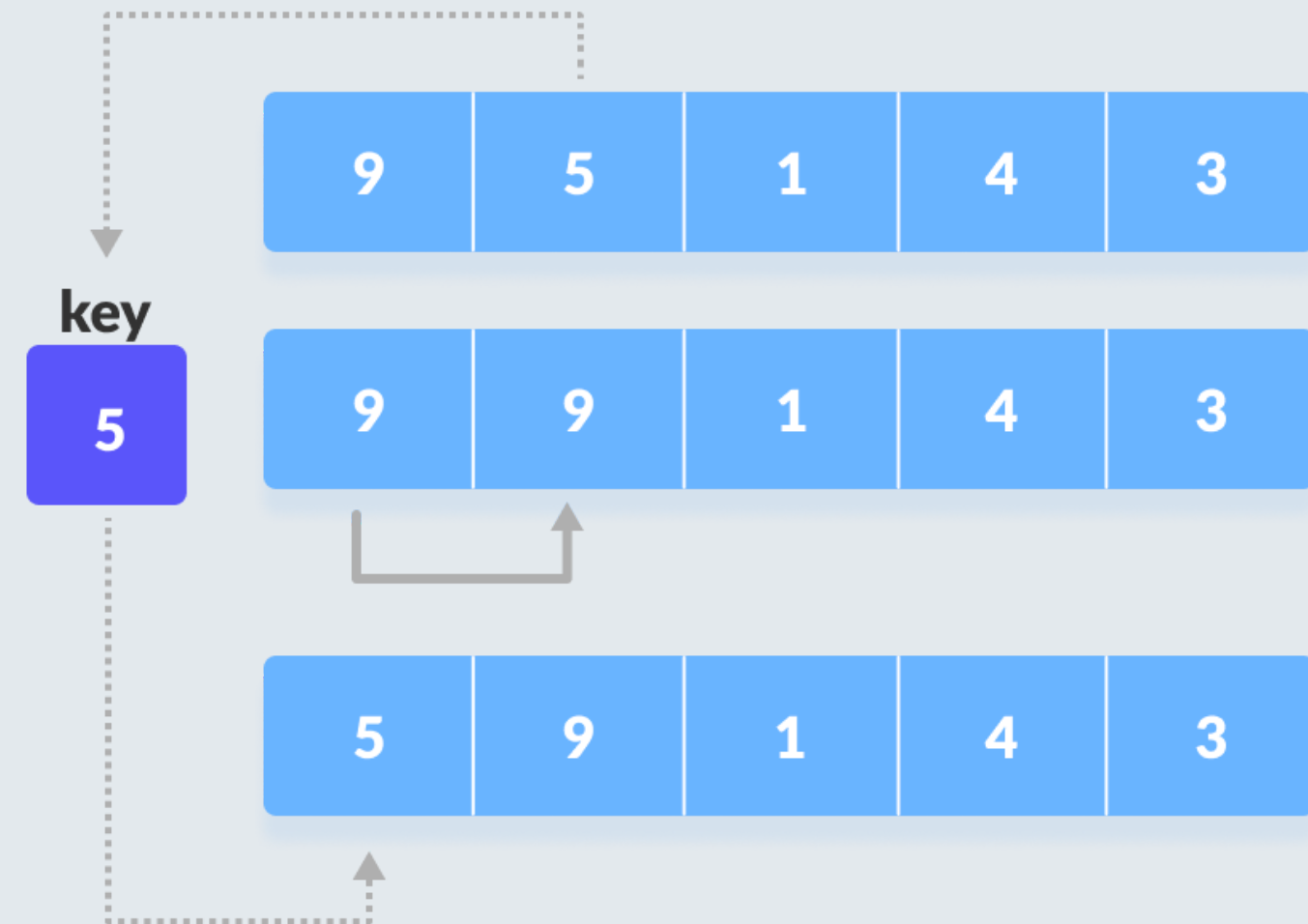


- Sortira niz postepeno, umećući svaki novi element na pravo mjesto u već sortiranom dijelu.
- Prvi element se smatra sortiranim; svaki naredni se poredi unazad i ubacuje gdje pripada.
- Rad algoritma podsjeća na ručno sortiranje karata u ruci.
- Jednostavan za implementaciju, ali nije pogodan za velike nizove
- Vremenska složenost: $O(n^2)$

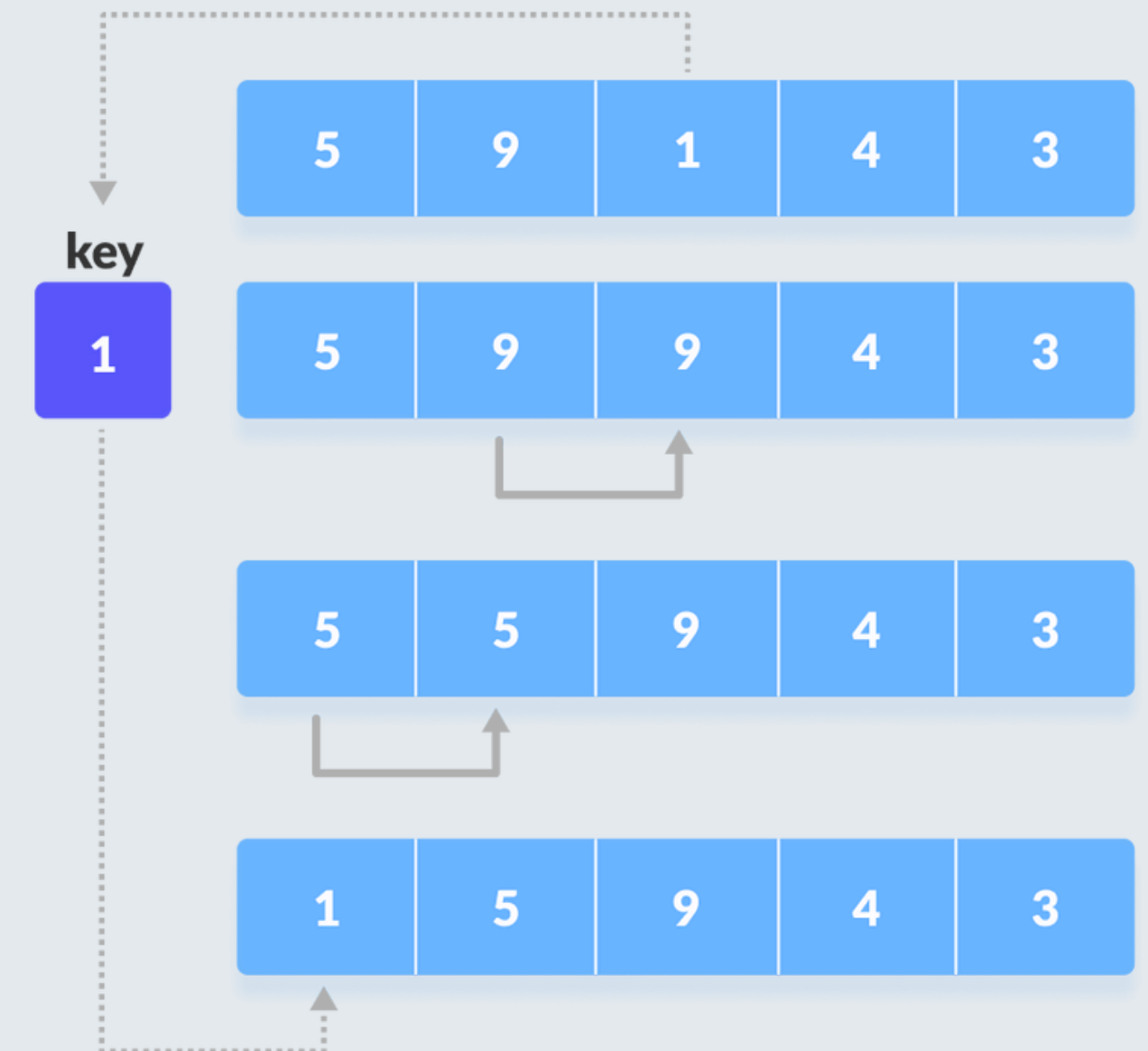




step = 1

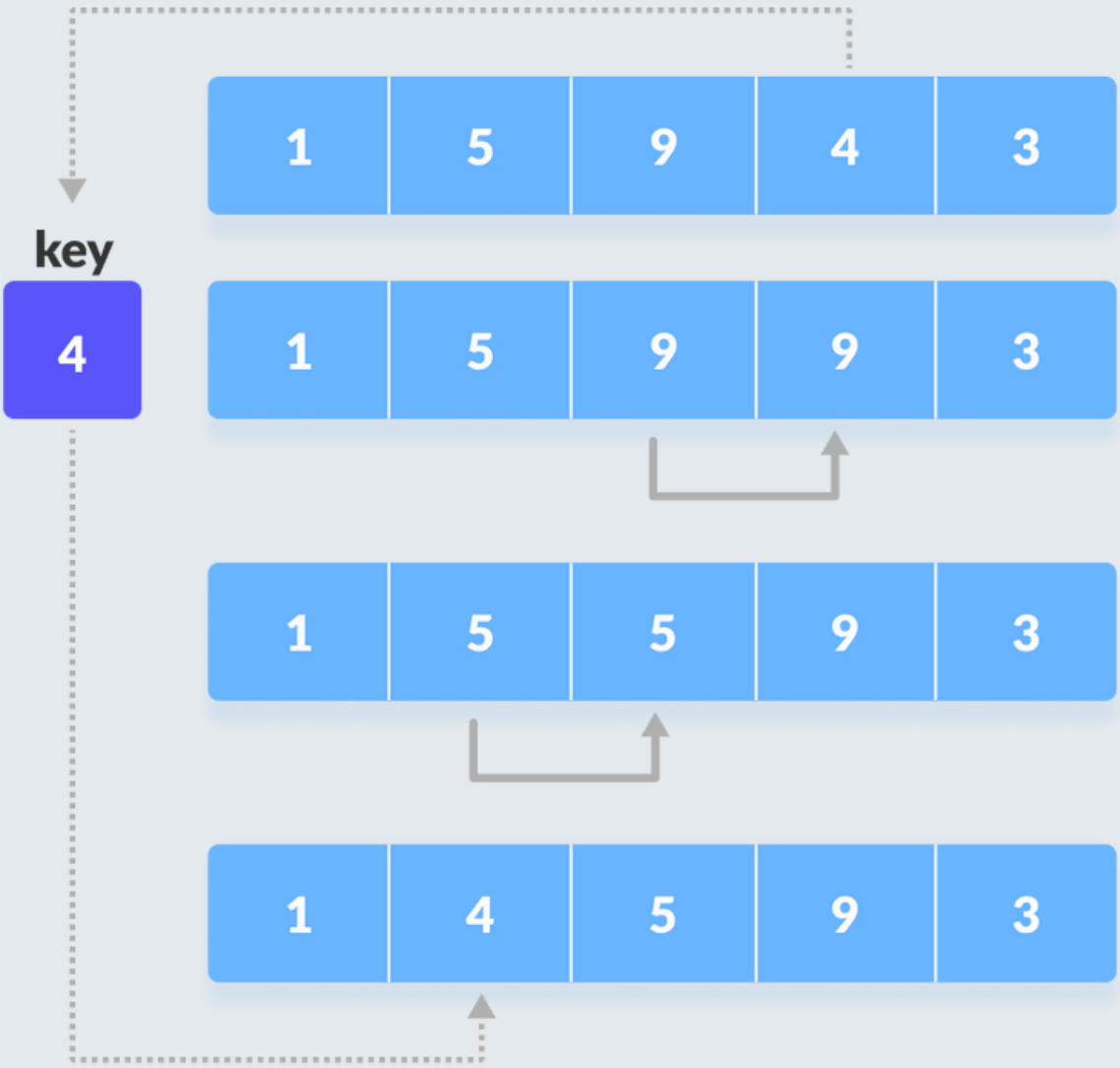


step = 2

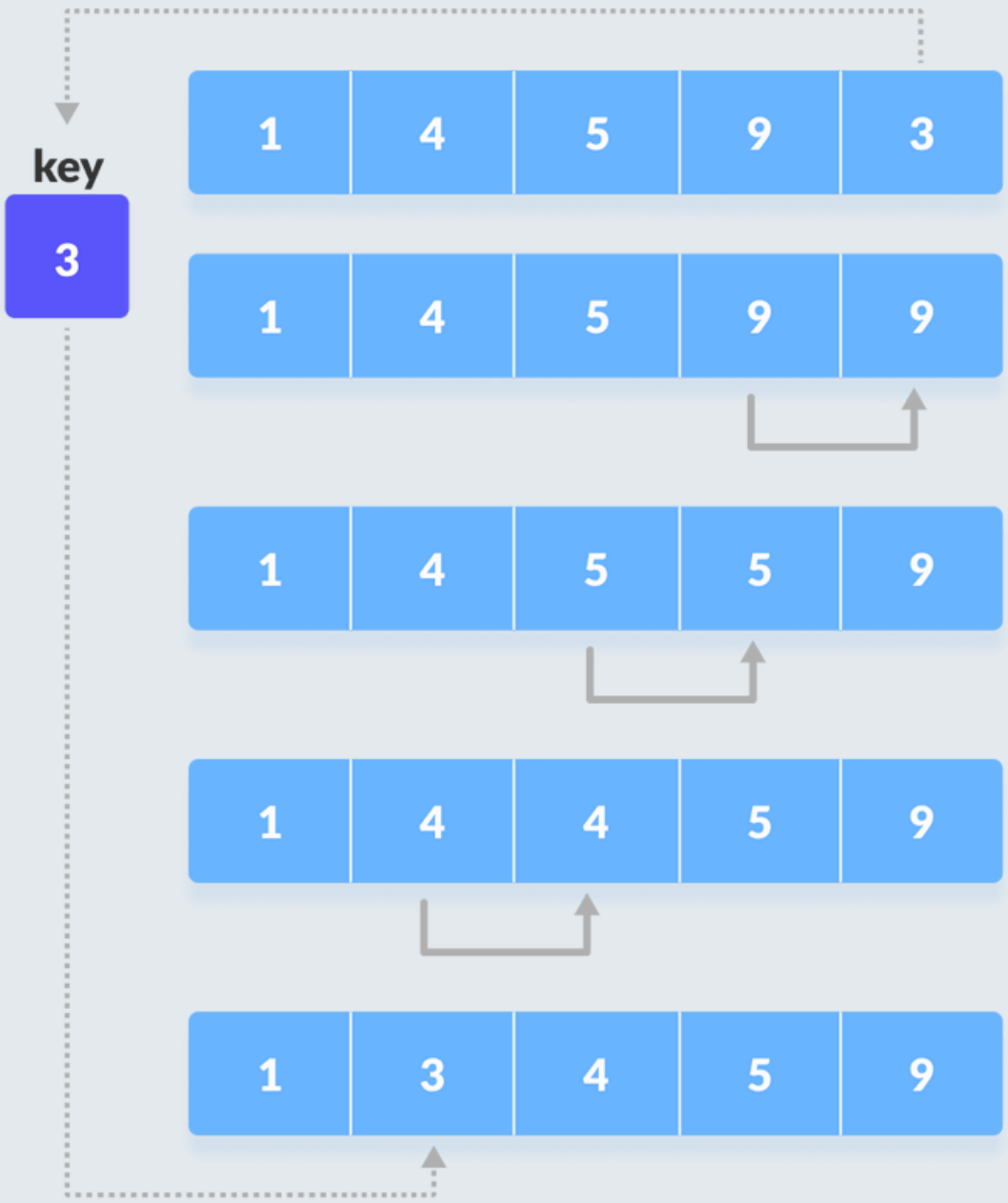




step = 3



step = 4



Sekvencijalni algoritam



```
#include <iostream>
#include <vector>

void insertionSort(std::vector<int>& arr) {
    int n = arr.size();
    for (int i = 1; i < n; ++i) {
        int key = arr[i];
        int j = i - 1;

        // Pomjeranje elemenata većih od ključa
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}
```

Paralelni algoritam



```
#include <omp.h>
#include <algorithm>

void selectionSortParallel(int* arr, int n) {
    for (int i = 0; i < n - 1; ++i) {
        int min_idx = i;

        #pragma omp parallel
        {
            int local_min_idx = min_idx;

            #pragma omp for nowait
            for (int j = i + 1; j < n; ++j) {
                if (arr[j] < arr[local_min_idx]) {
                    local_min_idx = j;
                }
            }

            #pragma omp critical
            {
                if (arr[local_min_idx] < arr[min_idx]) {
                    min_idx = local_min_idx;
                }
            }
        }

        std::swap(arr[i], arr[min_idx]);
    }
}
```

ANALIZA REZULTATA

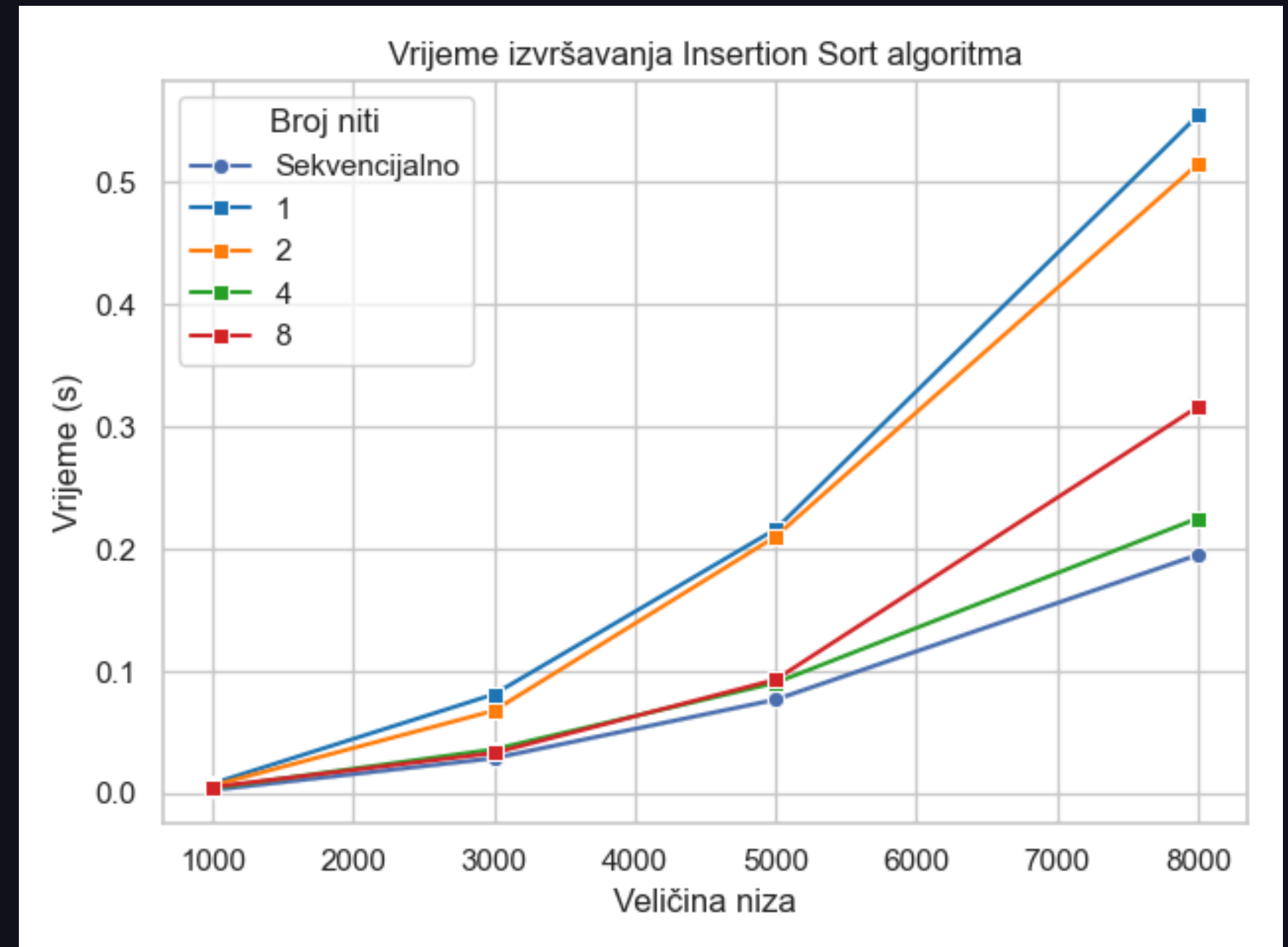


- Paralelizacija ne donosi poboljšanje performansi, čak ni za veće ulaze.
- Speedup ne prelazi 1 – maksimalno zabilježeno je 0.8735 (n = 3000, 8 niti).
- Efikasnost opada sa brojem niti, najniža je 0.0684 (n = 1000, 8 niti).
- Cost raste sa brojem niti – maksimalna vrijednost 2.5312 (n = 8000, 8 niti).
- Sekvencijalna verzija je stabilnija i efikasnija u svim slučajevima.
- Paralelizacija Insertion Sort-a uvodi više režije nego koristi zbog zavisnosti među iteracijama.

n	threads	seq_time	par_time	speedup	efficiency	cost
1000	1	0.002958	0.0082	0.360731	0.360731	0.0082
1000	2	0.002958	0.0066	0.448182	0.224091	0.0132
1000	4	0.002958	0.0044	0.67227	0.168068	0.0176
1000	8	0.002958	0.0054	0.547779	0.068472	0.0432
3000	1	0.029001	0.0812	0.35716	0.35716	0.0812
3000	2	0.029001	0.0678	0.427749	0.213875	0.1356
3000	4	0.029001	0.0362	0.801144	0.200286	0.1448
3000	8	0.029001	0.0332	0.873537	0.109192	0.2656
5000	1	0.076918	0.2162	0.355773	0.355773	0.2162
5000	2	0.076918	0.2102	0.365929	0.182964	0.4204
5000	4	0.076918	0.0906	0.848987	0.212247	0.3624
5000	8	0.076918	0.0932	0.825302	0.103163	0.7456
8000	1	0.195392	0.5546	0.352312	0.352312	0.5546
8000	2	0.195392	0.515	0.379402	0.189701	1.03
8000	4	0.195392	0.2252	0.867638	0.216909	0.9008
8000	8	0.195392	0.3164	0.617547	0.077193	2.5312

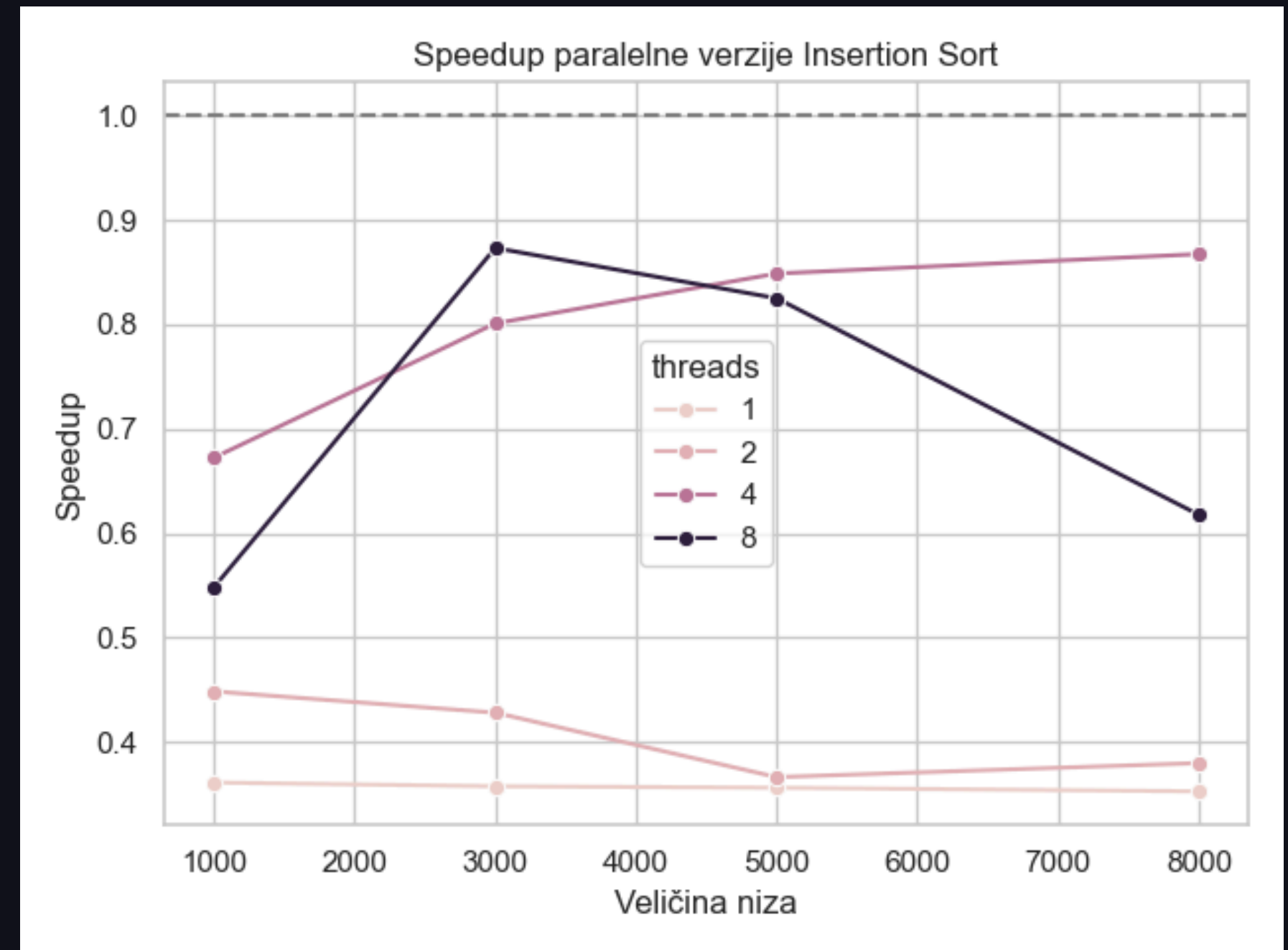
ANALIZA REZULTATA

- Sekvencijalna verzija ima najkraće vrijeme izvršavanja u svim slučajevima.
- 1 i 2 niti su sporije od sekvencijalne verzije.
- 4 i 8 niti daju blago poboljšanje, ali nikada ne nadmašuju sekvencijalnu verziju.
- Povećanje broja niti ne dovodi do proporcionalnog smanjenja vremena.
- Zavisnost među iteracijama čini paralelizaciju neefikasnom.



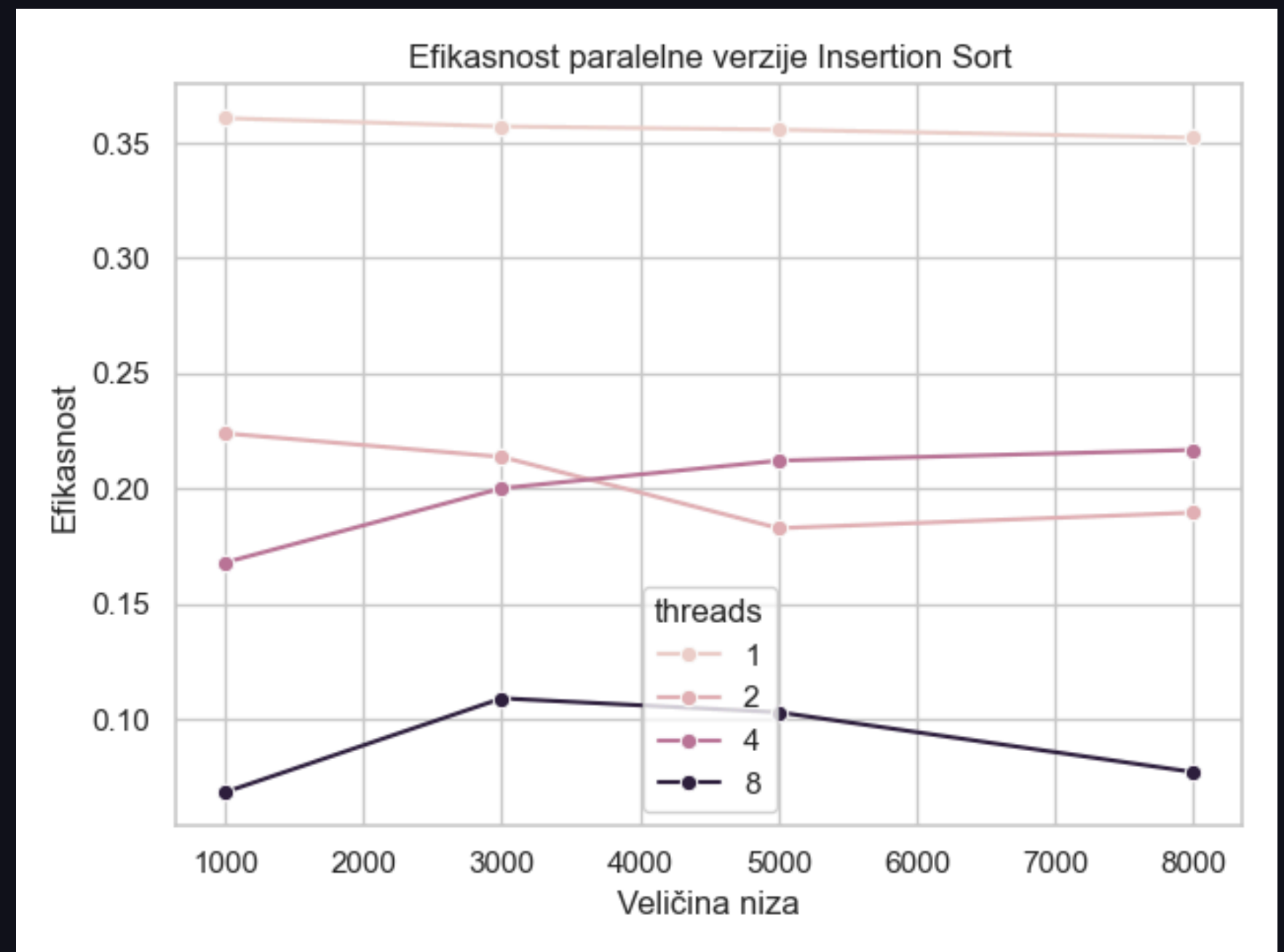
ANALIZA REZULTATA

- Speedup nikada ne prelazi 1 → paralelna verzija nije brža od sekvencijalne.
- Najveći speedup (~ 0.87) ostvaren za $n = 3000$ i 8 niti.
- 1 i 2 niti daju najmanji speedup, što ukazuje na dominaciju režije nad dobiti.
- Paralelizacija nije opravdana zbog minimalnog ubrzanja i dodatnog overhead-a.



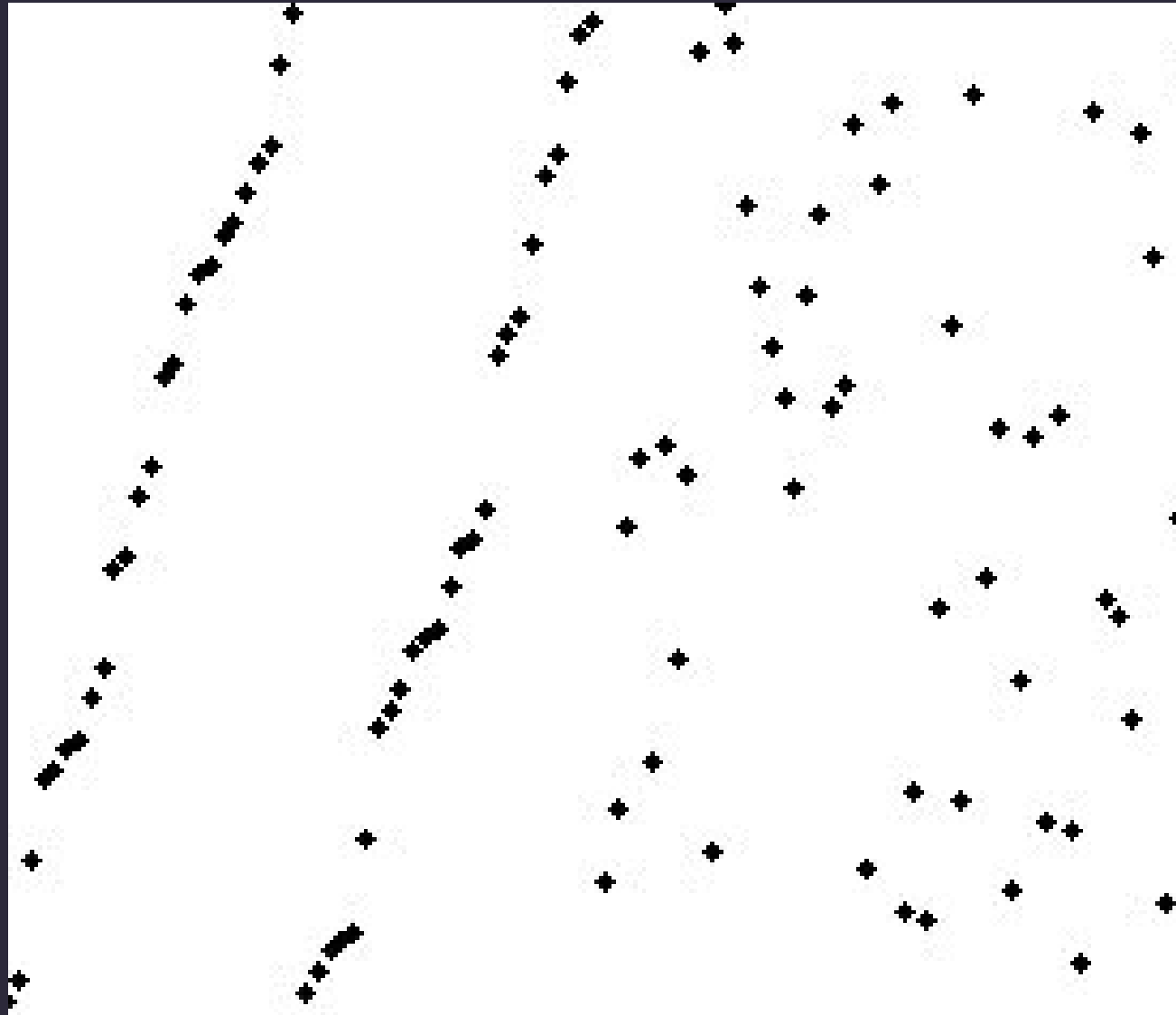
ANALIZA REZULTATA

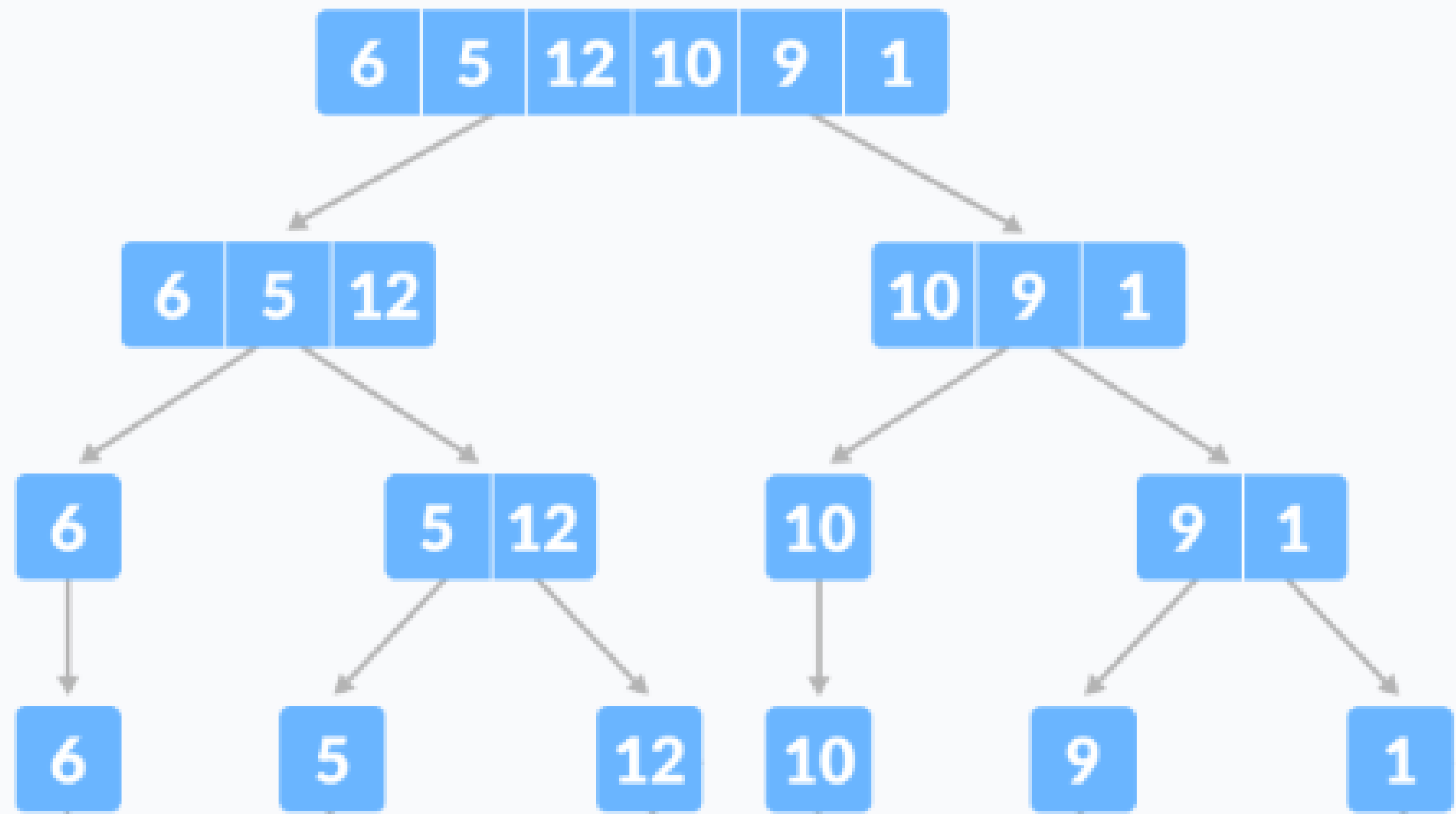
- Efikasnost opada sa više niti, naročito za 8 niti (ispod 0.1).
- Najefikasnija je verzija sa 1 niti – ekvivalent sekvencijalnoj obradi.
- Višestruke niti ne daju proporcionalnu dobit, već povećavaju trošak.
- Paralelna verzija nije skalabilna zbog zavisnosti među iteracijama.
- Eksperimenti potvrđuju: Insertion sort nije pogodan za paralelizaciju.

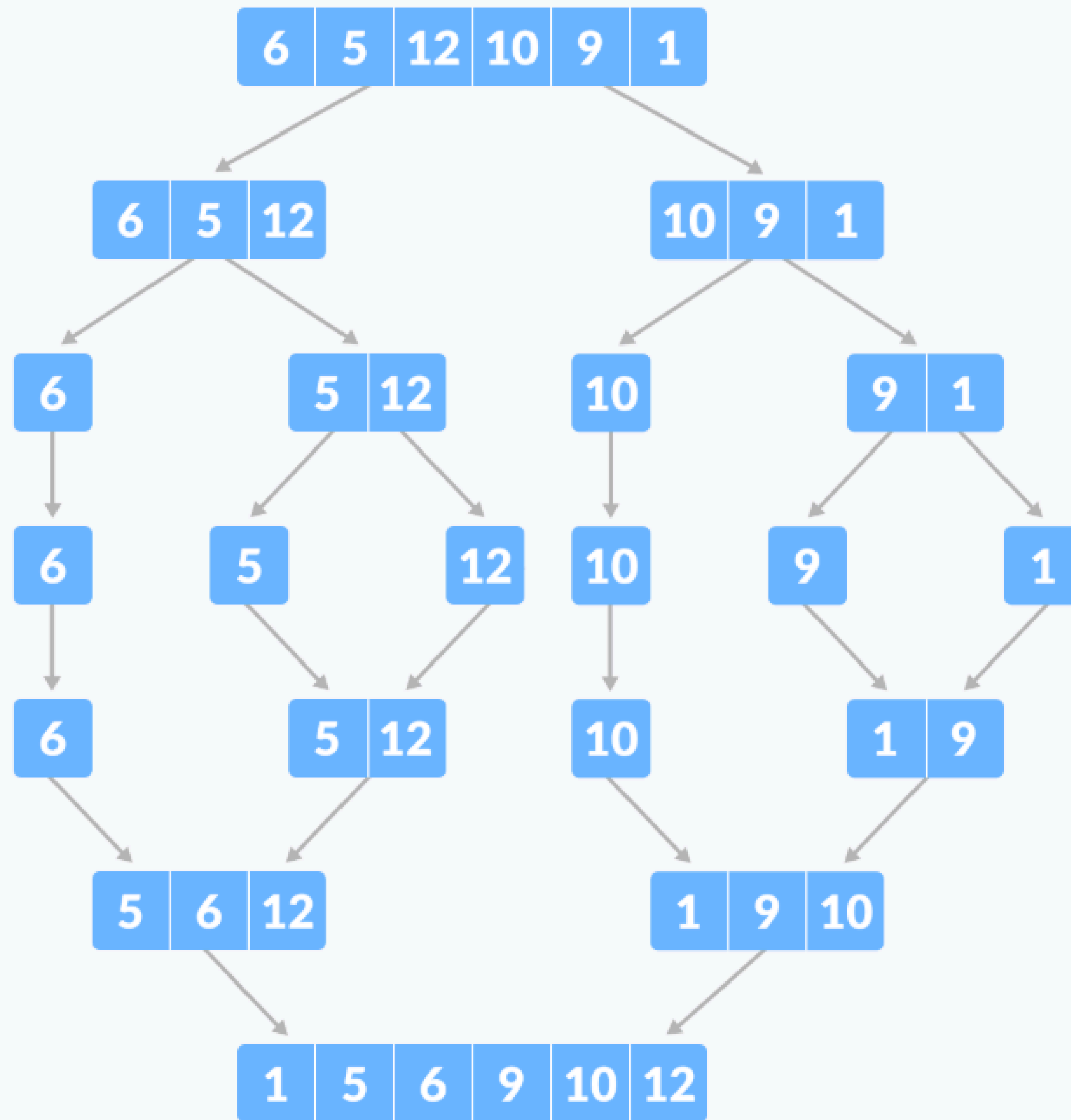


MERGE SORT

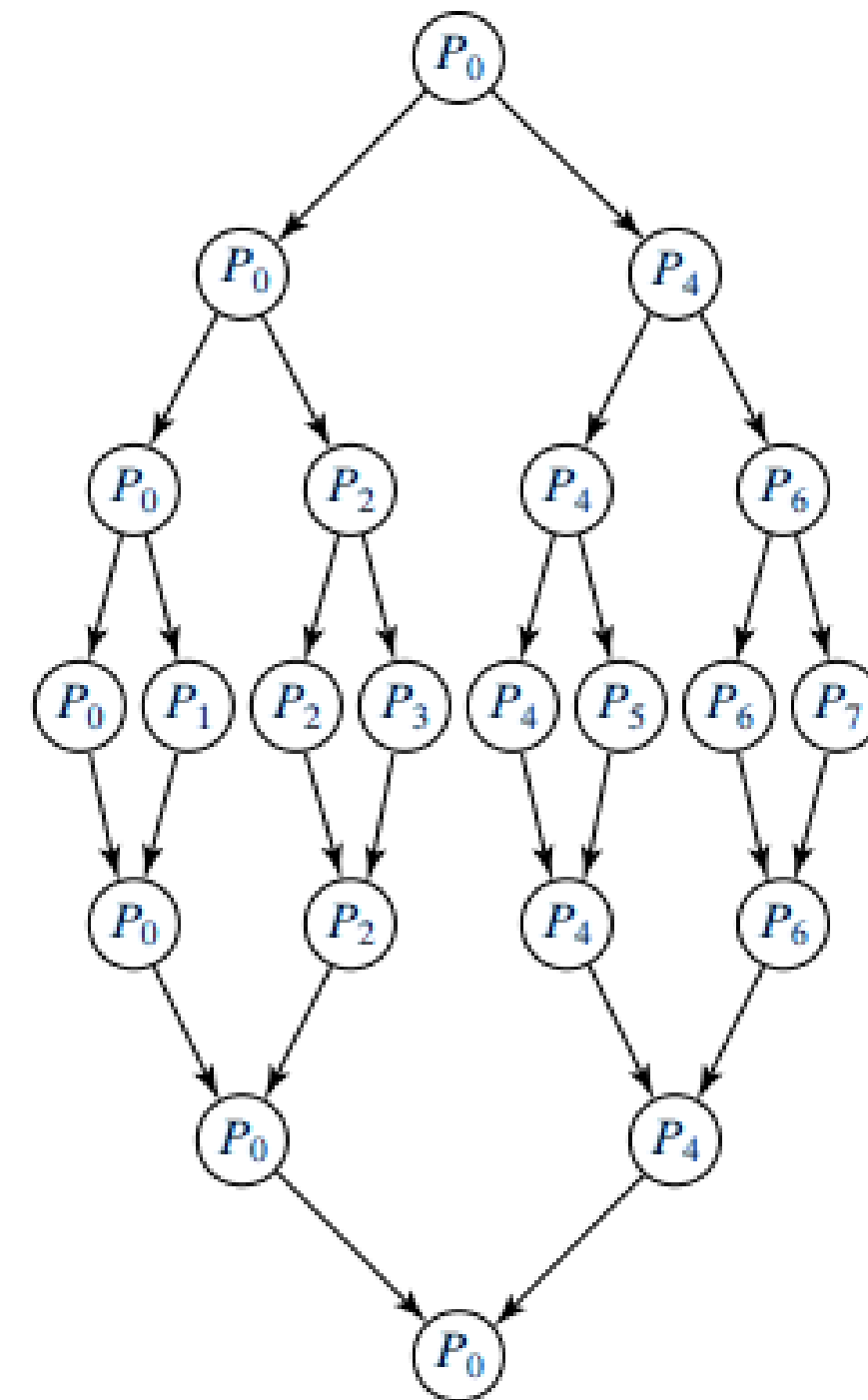
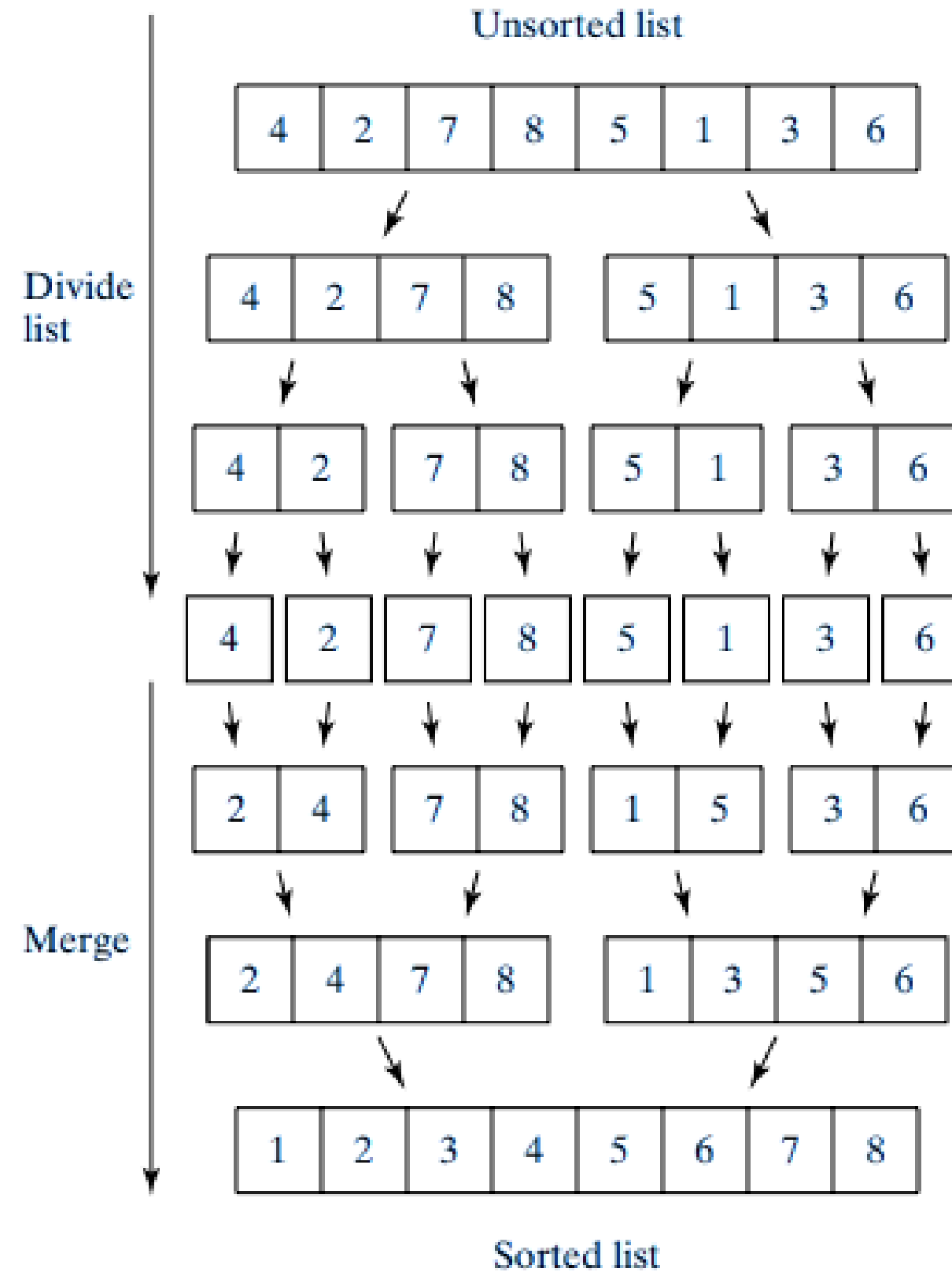
- Zasniva se na strategiji "Podijeli i vladaj" – problem se dijeli na manje dijelove koji se rekurzivno rješavaju i zatim spajaju.
- Tri glavna koraka: podjela niza, rekurzivno sortiranje i spajanje u veće sortirane cjeline.
- Stabilan algoritam – čuva redosljed jednakih elemenata.
- Nije in-place – koristi dodatni memorijski prostor tokom spajanja.







Ideja paralelizacije



Process allocation

Sekvencijalni algoritam

```
void mergeSort(std::vector<int>& arr, int left, int right)
{
    if(left >= right) return;

    int mid = left + (right - left) / 2;

    mergeSort(arr, left, mid);
    mergeSort(arr, mid + 1, right);

    merge(arr, left, mid, right);
}
```



```
void merge(std::vector<int>& arr, int left, int mid, int right) {  
    std::vector<int> temp(right - left + 1);  
    int i = left, j = mid + 1, k = 0;  
  
    while(i <= mid && j <= right) {  
        if(arr[i] <= arr[j])  
            temp[k++] = arr[i++];  
        else  
            temp[k++] = arr[j++];  
    }  
  
    while(i <= mid) temp[k++] = arr[i++];  
    while(j <= right) temp[k++] = arr[j++];  
  
    for(int t = 0; t < temp.size(); ++t)  
        arr[left + t] = temp[t];  
}
```

Paralelni algoritam

```
void parallelMergeSort(std::vector<int>& arr, int left, int right, int depth = 0) {  
    if(left >= right) return;  
  
    int mid = left + (right - left) / 2;  
  
    if(depth <= 4) { // ograničimo dubinu paralelizacije da se izbjegne previše niti  
        #pragma omp parallel sections  
        {  
            #pragma omp section  
            parallelMergeSort(arr, left, mid, depth + 1);  
  
            #pragma omp section  
            parallelMergeSort(arr, mid + 1, right, depth + 1);  
        }  
    } else {  
        parallelMergeSort(arr, left, mid, depth + 1);  
        parallelMergeSort(arr, mid + 1, right, depth + 1);  
    }  
  
    merge(arr, left, mid, right);  
}
```

ANALIZA REZULTATA

- **Paralelizacija nije efikasna za male ulaze** (npr. za $n = 1000$, speedup za 4 niti iznosi samo 0.6605, a efikasnost 0.1651)
- **Prekomjerno paralelizovanje može biti kontraproduktivno** (najniži speedup 0.4521 i efikasnost 0.0565 javljaju se pri 8 tredova i $n = 20000$)
- **Za srednje do veće ulaze ($n = 5000$ – 10000), paralelizacija pokazuje umjeren dobitak** (najveći speedup 1.5988 ostvaren je za $n = 10000$ sa 4 tred-a)
- **Više tredova $\neq$ nužno bolje performanse** (npr. za $n = 20000$, 2 tred-a daju bolju efikasnost 0.2260 nego 8 tredova 0.1200)

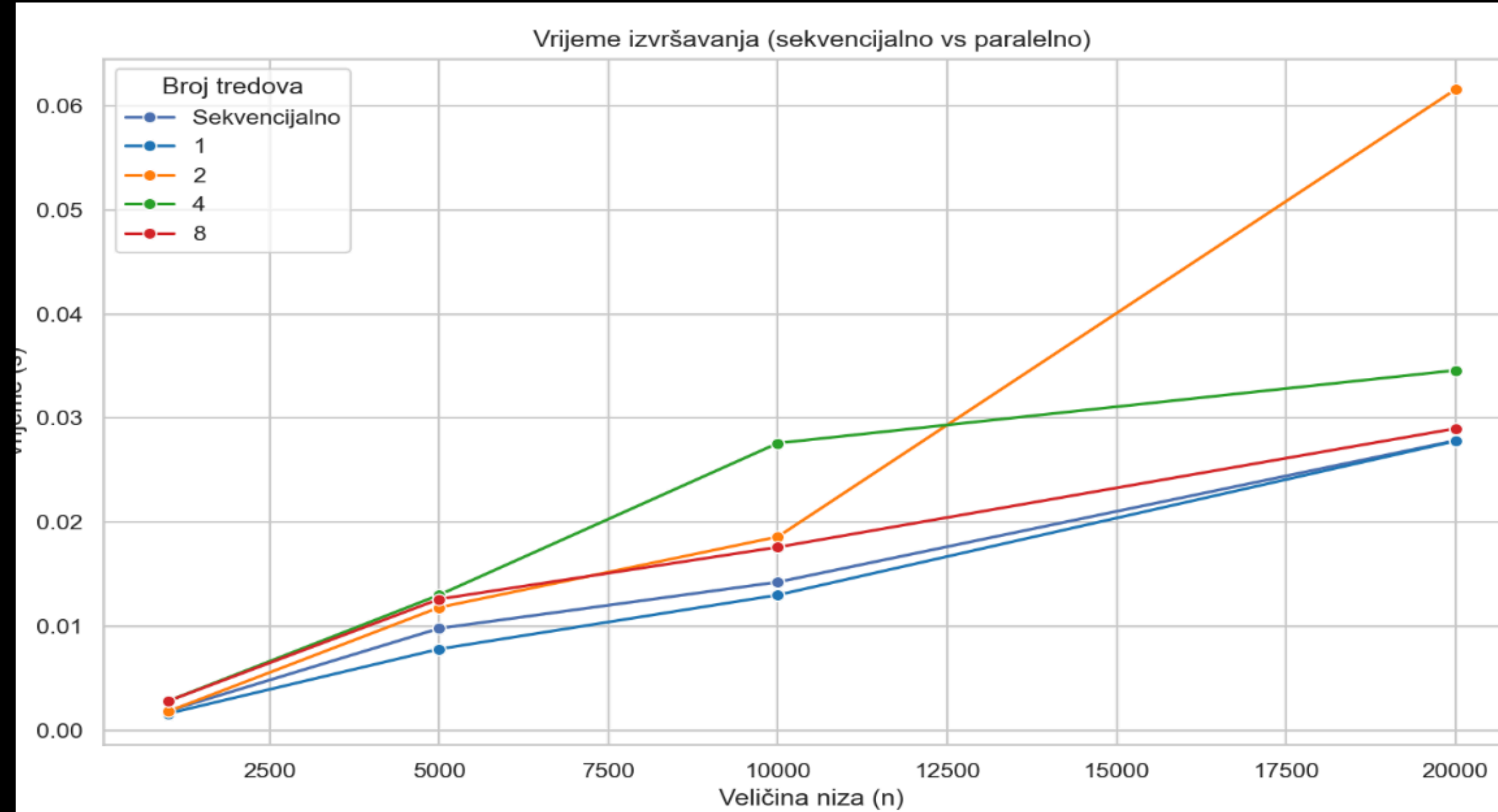
n	threads	seq_time	par_time	speedup	efficiency	cost
1000.0	1.0	0.001849	0.0016	1.155893	1.0	0.0016
1000.0	2.0	0.001849	0.0018	1.027471	0.513735	0.0036
1000.0	4.0	0.001849	0.0028	0.660524	0.165131	0.0112
1000.0	8.0	0.001849	0.0028	0.660524	0.082566	0.0224
5000.0	1.0	0.009789	0.0078	1.254935	1.0	0.0078
5000.0	2.0	0.009789	0.0118	0.829534	0.414767	0.0236
5000.0	4.0	0.009789	0.013	0.752961	0.18824	0.052
5000.0	8.0	0.009789	0.0126	0.776863	0.097108	0.1008
10000.0	1.0	0.014246	0.013	1.095881	1.0	0.013
10000.0	2.0	0.014246	0.0186	0.765937	0.382969	0.0372
10000.0	4.0	0.014246	0.0276	0.516176	0.129044	0.1104
10000.0	8.0	0.014246	0.0176	0.809457	0.101182	0.1408
20000.0	1.0	0.027848	0.0278	1.00174	1.0	0.0278
20000.0	2.0	0.027848	0.0616	0.452085	0.226042	0.1232
20000.0	4.0	0.027848	0.0346	0.804867	0.201217	0.1384
20000.0	8.0	0.027848	0.029	0.960289	0.120036	0.232

Zaključak:

Merge Sort posjeduje umjerenu skalabilnost, ali koristi od paralelizacije zavise od veličine ulaza i optimalnog broja tredova. Prevelik broj tredova može povećati trošak i smanjiti efikasnost.

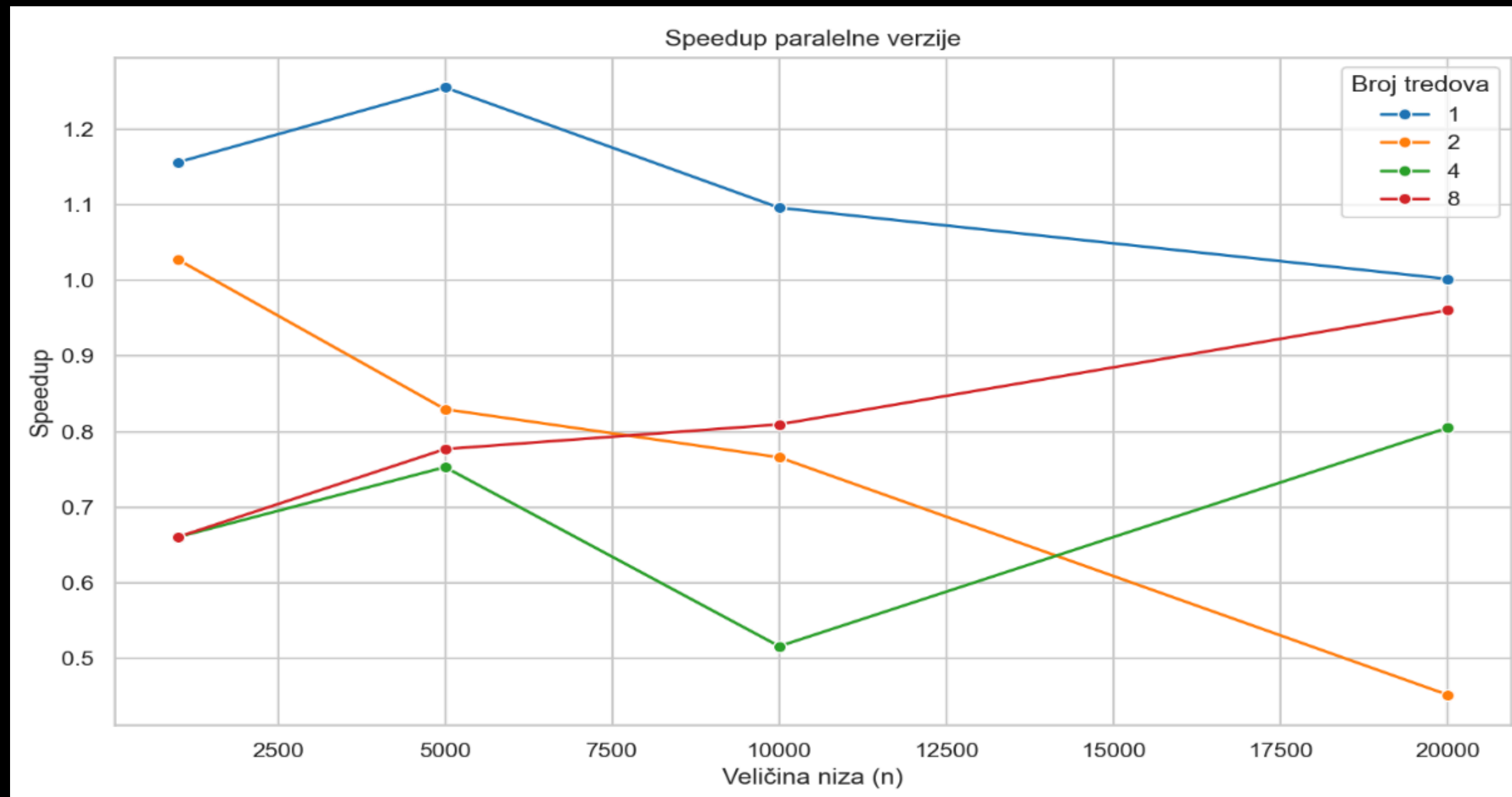
ANALIZA REZULTATA

- Vrijeme izvršavanja raste sa veličinom niza kod svih verzija algoritma.
- Paralelna verzija sa 2 niti najsporija – čak sporija od sekvencijalne za veće ulaze.
- Sekvencijalna i verzije sa 1 i 8 tredova pokazuju stabilne performanse.
- Paralelizacija ne daje benefite za male nizove, a koristi se javljaju tek kod većih



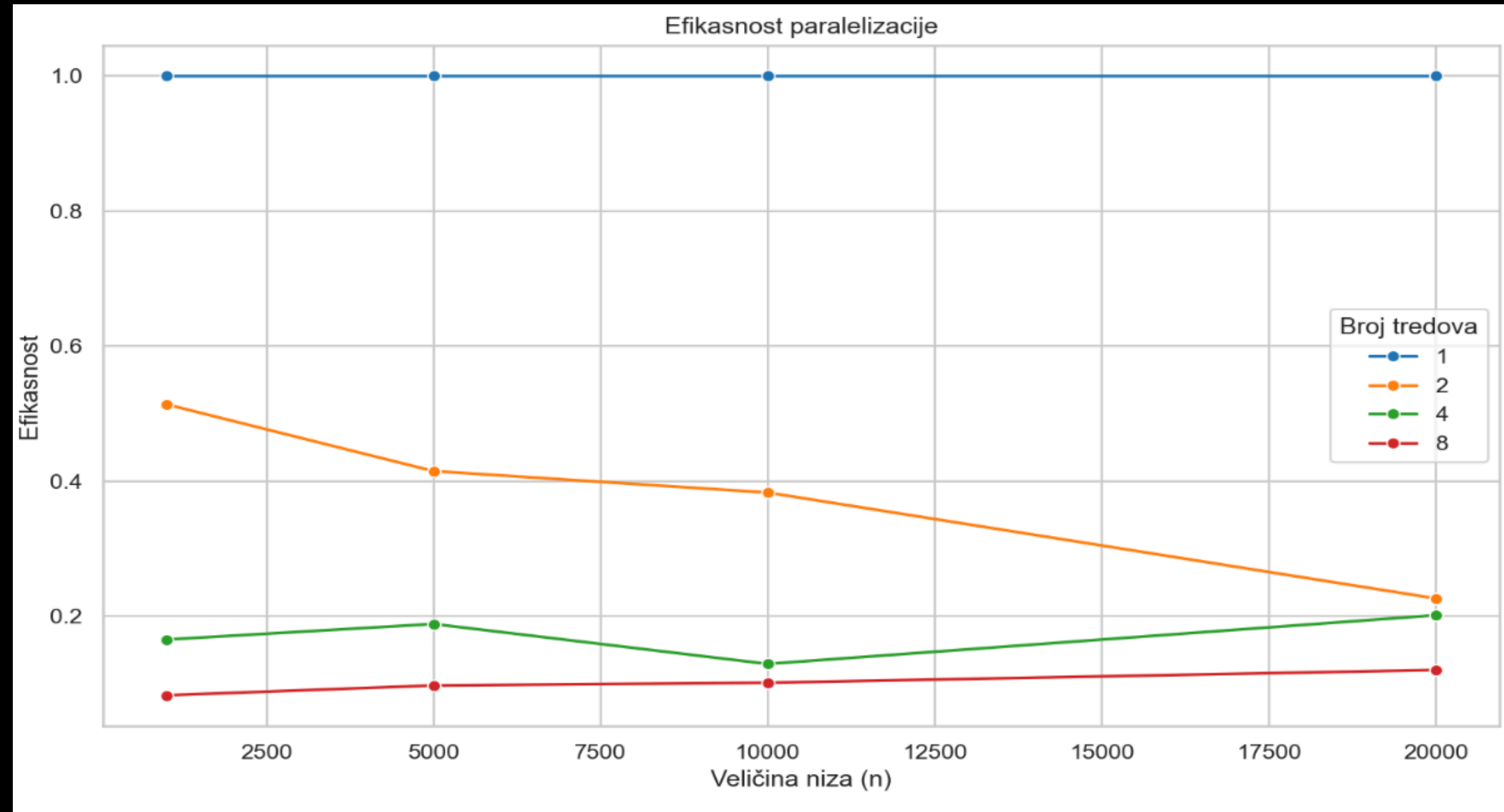
ANALIZA REZULTATA

- Najveći speedup (~1.25) ostvaren je sa 1 thread-om, što je zapravo sekvencijalni slučaj.
- Verzija sa 2 thread-a pokazuje pad performansi kako raste veličina niza (npr. speedup < 0.5 za n=20000).
- 4 i 8 thread-a ne postižu linearni speedup – benefiti se vide tek za veće ulaze.
- Speedup raste sporo i ostaje ispod 1 za većinu kombinacija broja tredova i veličina niza.

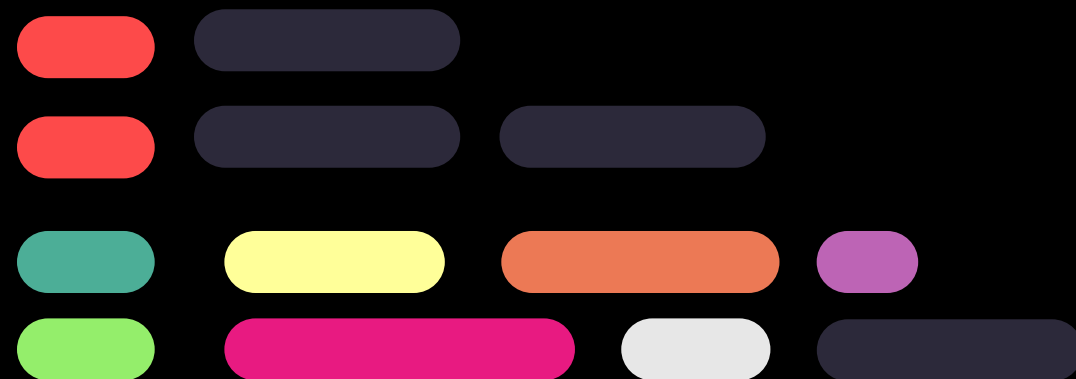
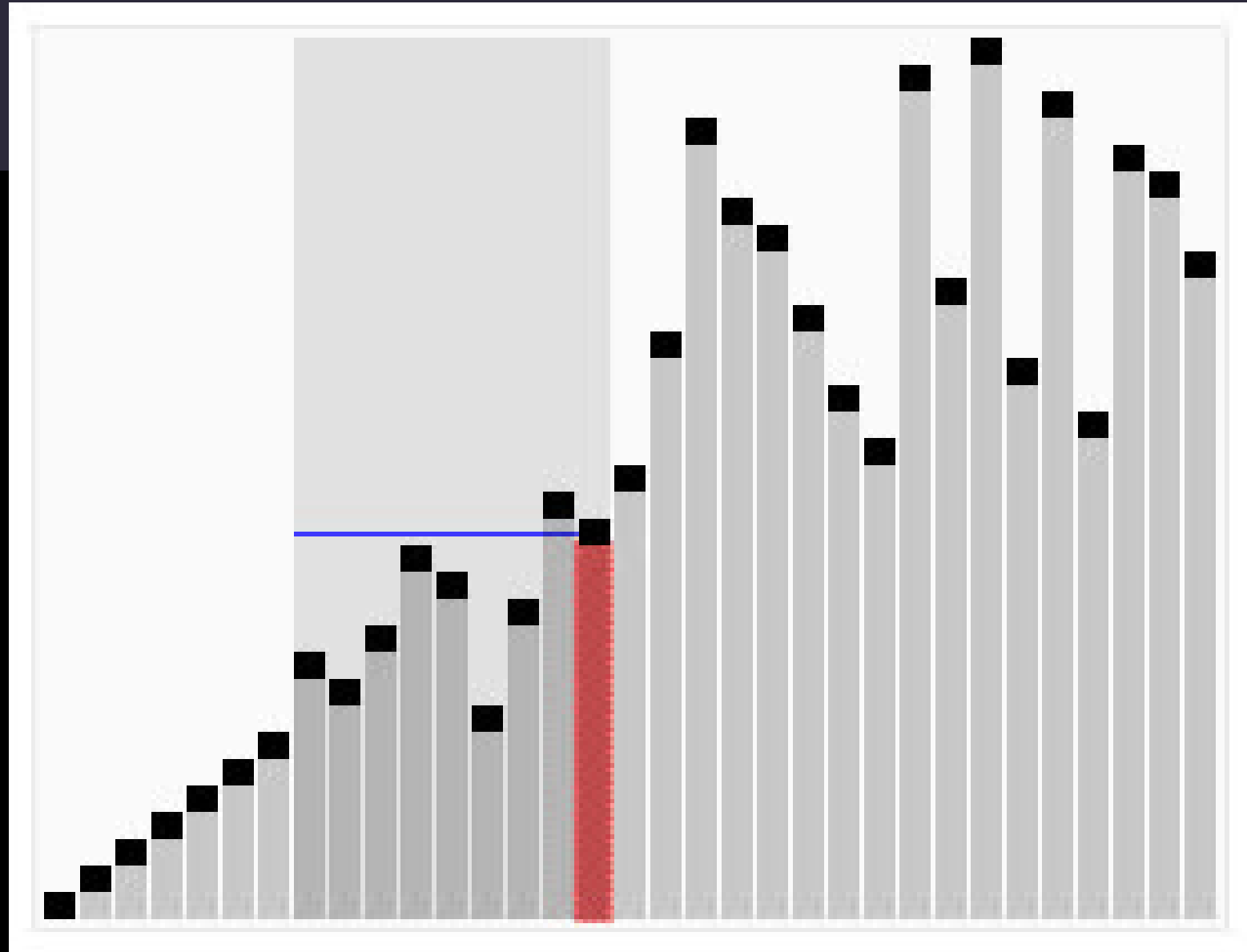


ANALIZA REZULTATA

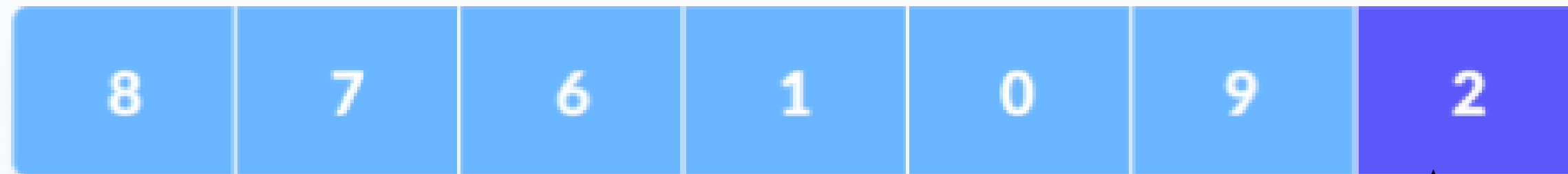
- Najviša efikasnost (1.0) dostiže se samo kod 1 thread-a — očekivano, jer nije prava paralelizacija.
- Efikasnost opada sa brojem tredova, naročito kod 8 thread-a (ispod 0.15 konstantno).
- Ni za velike nizove ($n=20000$) nema značajnog poboljšanja u efikasnosti.
- Dodavanje tredova ne donosi proporcionalan dobitak — što potvrđuje ograničenja u skalabilnosti algoritma.



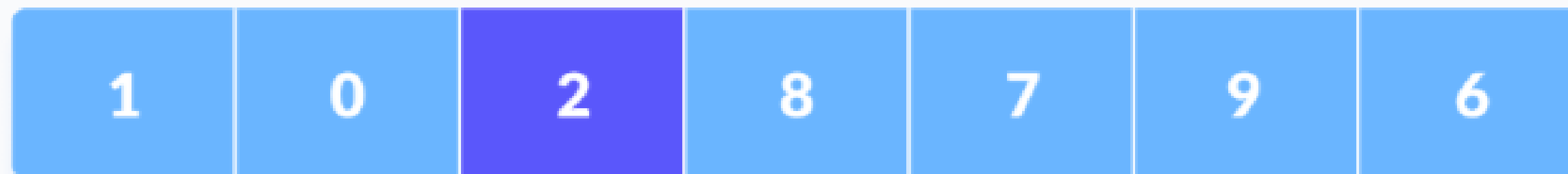
QUICK SORT



- Zasniva se na strategiji "Podijeli i vladaj" – koristi pivot da podijeli niz na manje dijelove.
- Pivot određuje razdvajanje – elementi manji od pivota idu lijevo, veći desno.
- Rekurzivno se primjenjuje na podnizove – svaki dio se ponovo sortira istim principom dok ne ostanu nizovi od jednog elementa.
- Sortira in-place – ne koristi dodatni prostor za sortiranje.
- Vrlo efikasan u praksi – brz i pogodan za rad sa velikim količinama podataka.

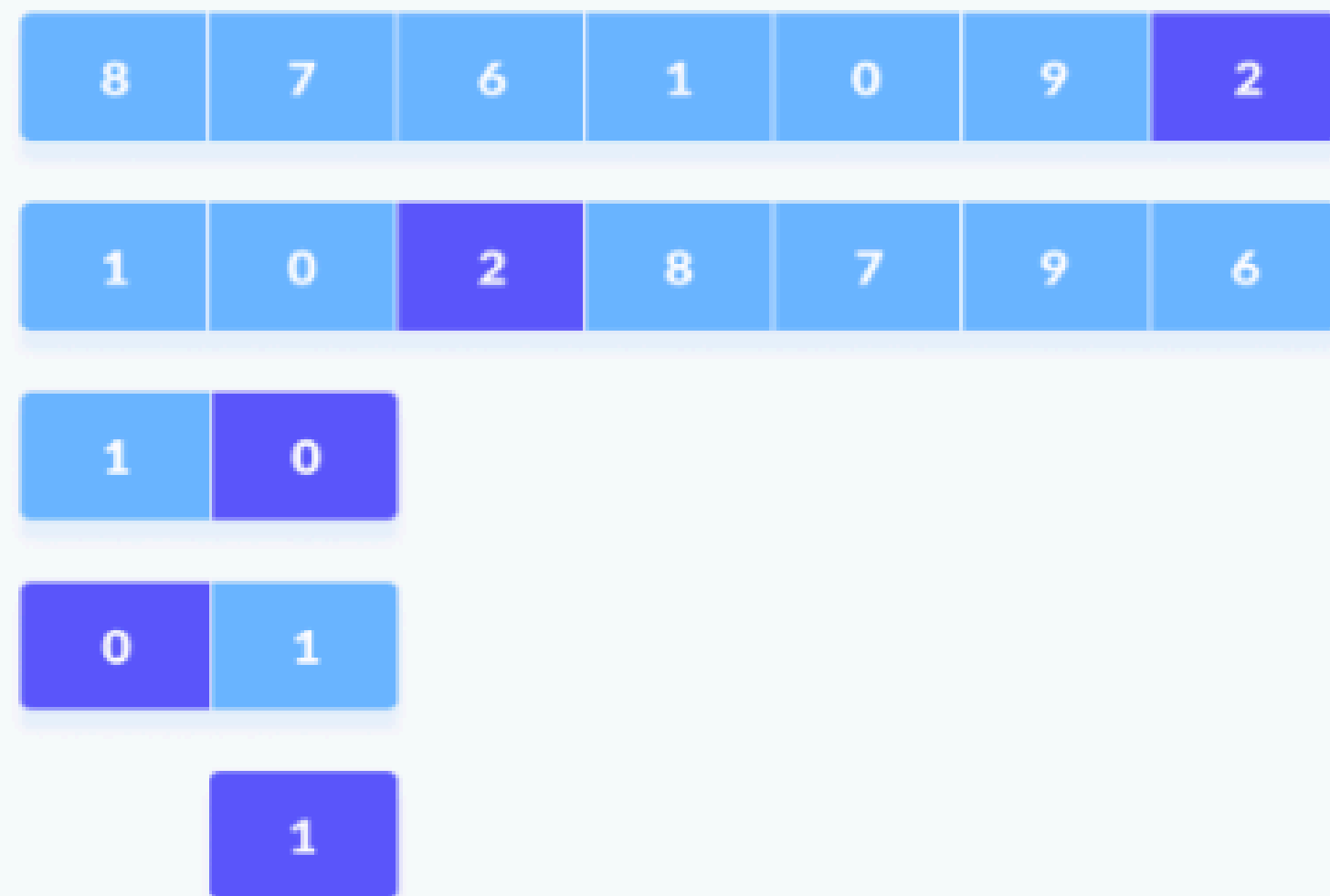


pivot

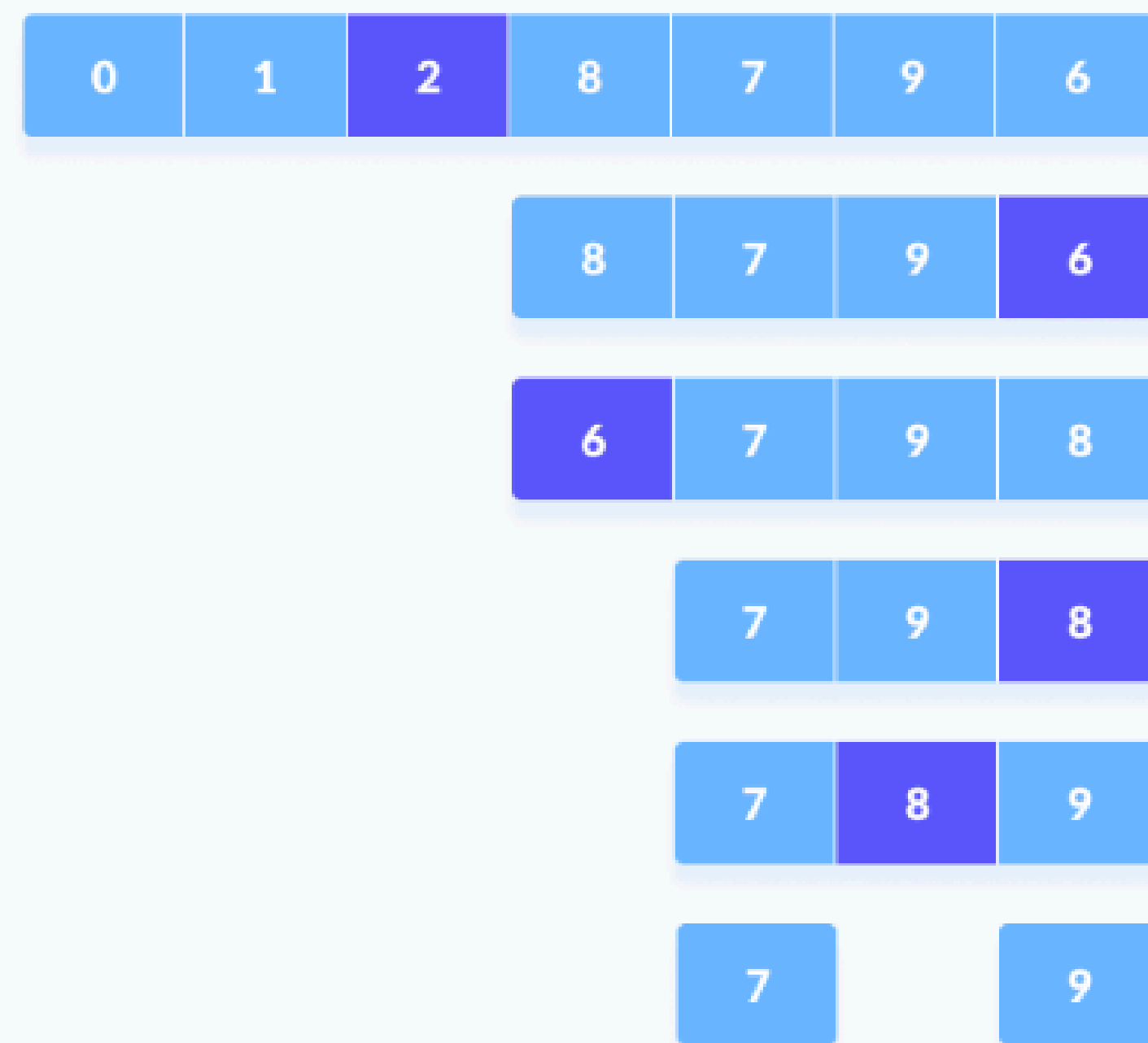


Cilj - lijevo od pivota manji, desni veći elementi

Rekurzivno sortiranje lijevog podniza



Rekurzivno sortiranje desnog podniza



Sekvencijalni algoritam

```
int partition(int arr[], int low, int high) {
    int pivot = arr[high]; // pivot je poslednji element
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1); // lijeva polovina
        quickSort(arr, pi + 1, high); // desna polovina
    }
}
```

Paralelni algoritam

```
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}
```

```
void quickSortParallel(int arr[], int low, int high, int depth = 0) {
    if (low < high) {
        int pi = partition(arr, low, high);

        // Ograniči paralelizaciju da se ne stvori previše niti
        if (depth < 4) {
            #pragma omp parallel sections
            {
                #pragma omp section
                quickSortParallel(arr, low, pi - 1, depth + 1);

                #pragma omp section
                quickSortParallel(arr, pi + 1, high, depth + 1);
            }
        } else {
            quickSortParallel(arr, low, pi - 1, depth + 1);
            quickSortParallel(arr, pi + 1, high, depth + 1);
        }
    }
}
```



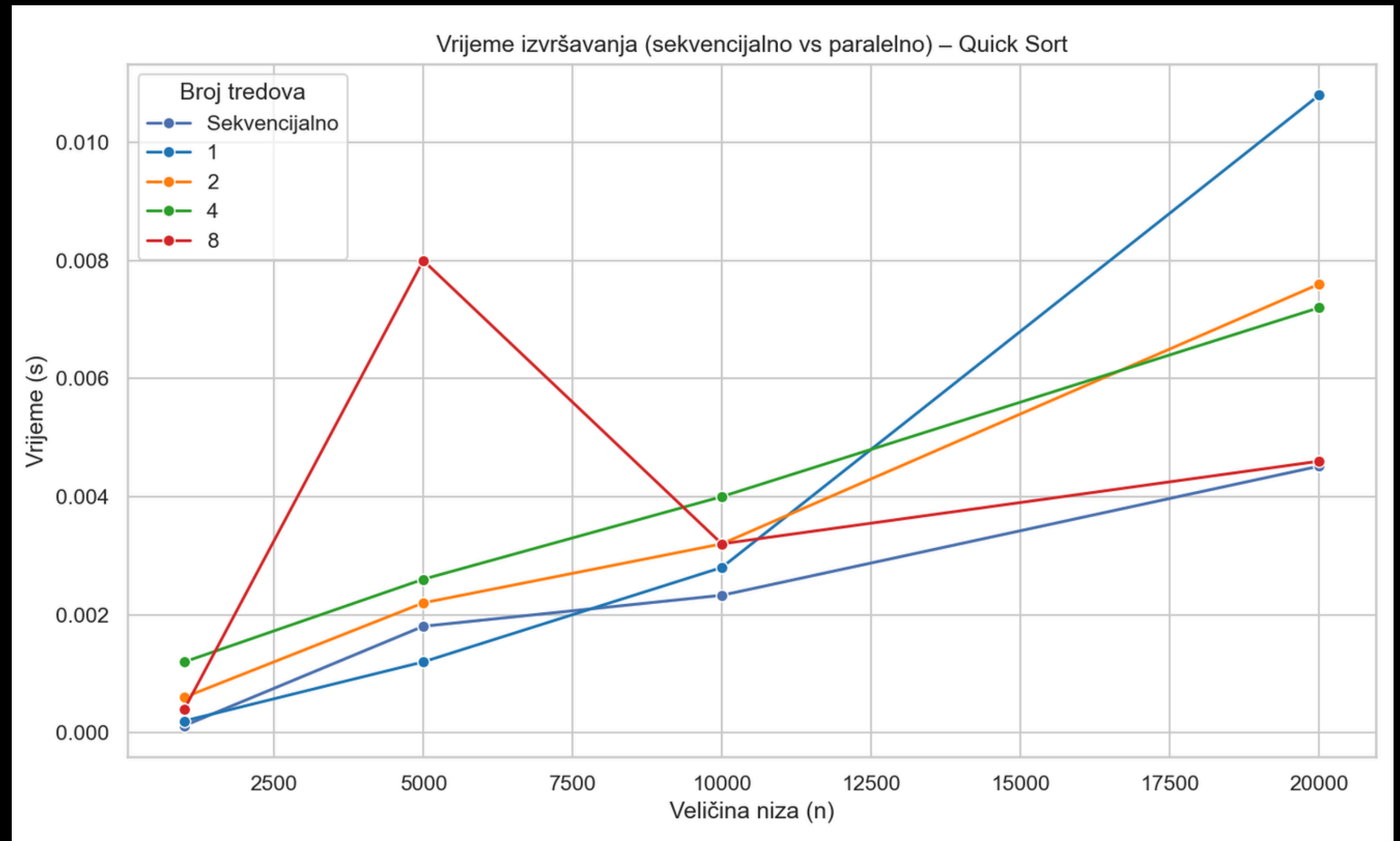
ANALIZA REZULTATA

- Najveći speedup (1.503) ostvaren je za $n = 5000$ i 1 thread, što ujedno predstavlja sekvencijalni slučaj.
- Efikasnost brzo opada sa povećanjem broja thread-ova, posebno kod manjih nizova (npr. ispod 0.05 za 8 thread-ova i $n = 1000$).
- Dodavanje više thread-ova ne donosi proporcionalan dobitak – speedup se rijetko povećava, dok cost značajno raste.
- Umjereni benefiti primjećuju se kod srednjih veličina ulaza ($n = 5000 - 10000$), ali bez jasnog trenda skalabilnosti.

n	threads	seq_time	par_time	speedup	efficiency	cost
1000.0	1.0	0.000121	0.0002	0.603844	0.603844	0.0002
1000.0	2.0	0.000121	0.0006	0.201265	0.100633	0.0012
1000.0	4.0	0.000121	0.0012	0.100633	0.025158	0.0048
1000.0	8.0	0.000121	0.0004	0.301922	0.03774	0.0032
5000.0	1.0	0.001804	0.0012	1.503316	1.0	0.0012
5000.0	2.0	0.001804	0.0022	0.81997	0.409985	0.0044
5000.0	4.0	0.001804	0.0026	0.693815	0.173454	0.0104
5000.0	8.0	0.001804	0.008	0.225492	0.028186	0.064
10000.0	1.0	0.002329	0.0028	0.831625	0.831625	0.0028
10000.0	2.0	0.002329	0.0032	0.727667	0.363834	0.0064
10000.0	4.0	0.002329	0.004	0.582129	0.145532	0.016
10000.0	8.0	0.002329	0.0032	0.727667	0.090958	0.0256
20000.0	1.0	0.004516	0.0108	0.418154	0.418154	0.0108
20000.0	2.0	0.004516	0.0076	0.594219	0.29711	0.0152
20000.0	4.0	0.004516	0.0072	0.627233	0.156808	0.0288
20000.0	8.0	0.004516	0.0046	0.981767	0.122721	0.0368

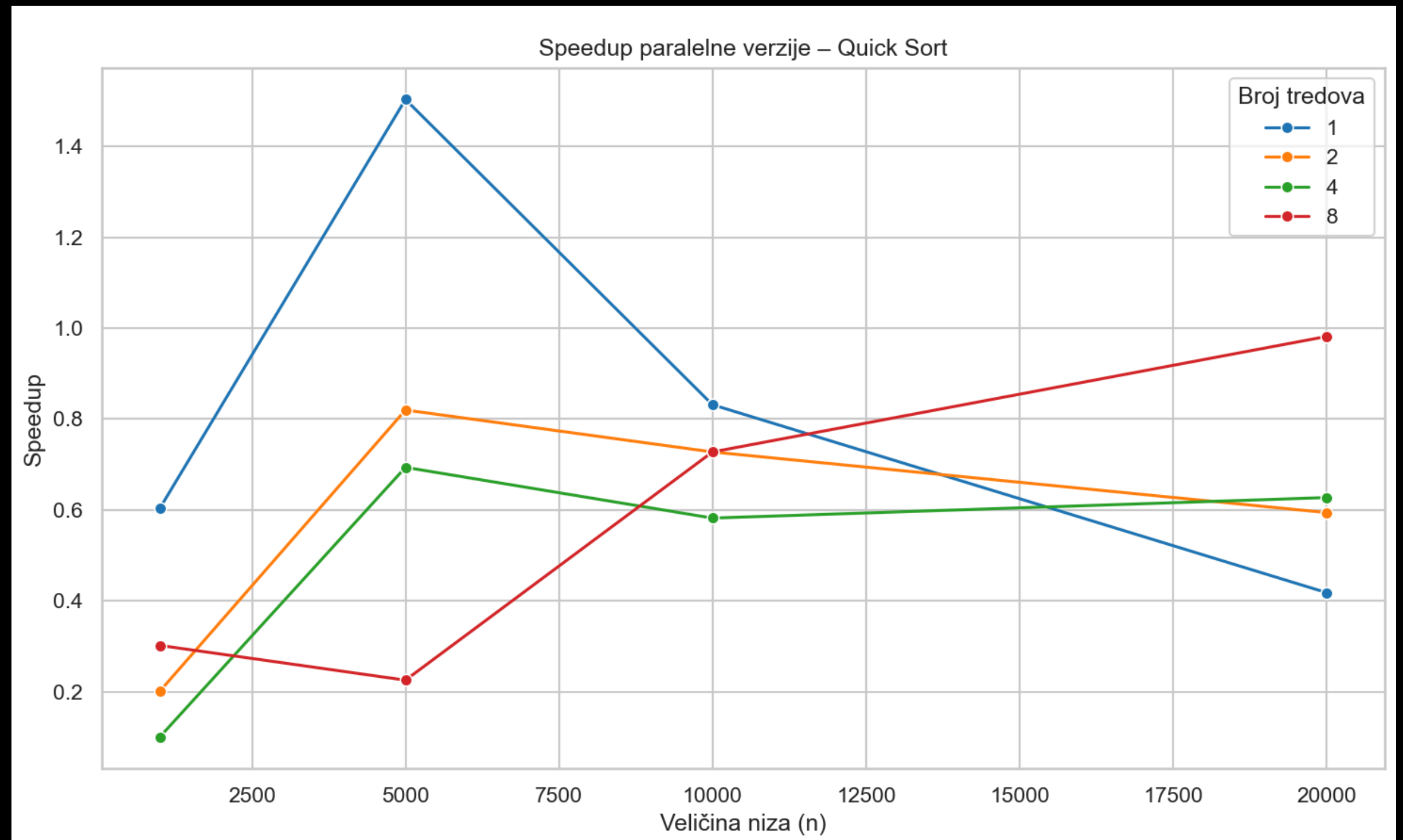
ANALIZA REZULTATA

- Vrijeme izvršavanja raste sa veličinom niza za sve verzije.
- Sekvencijalna verzija postaje najsporija tek kod $n = 20000$.
- Verzija sa 8 thread-ova pokazuje najveću nestabilnost (nagli skok za $n = 5000$).
- Paralelizacija postaje korisna tek kod većih nizova ($n \geq 10000$).



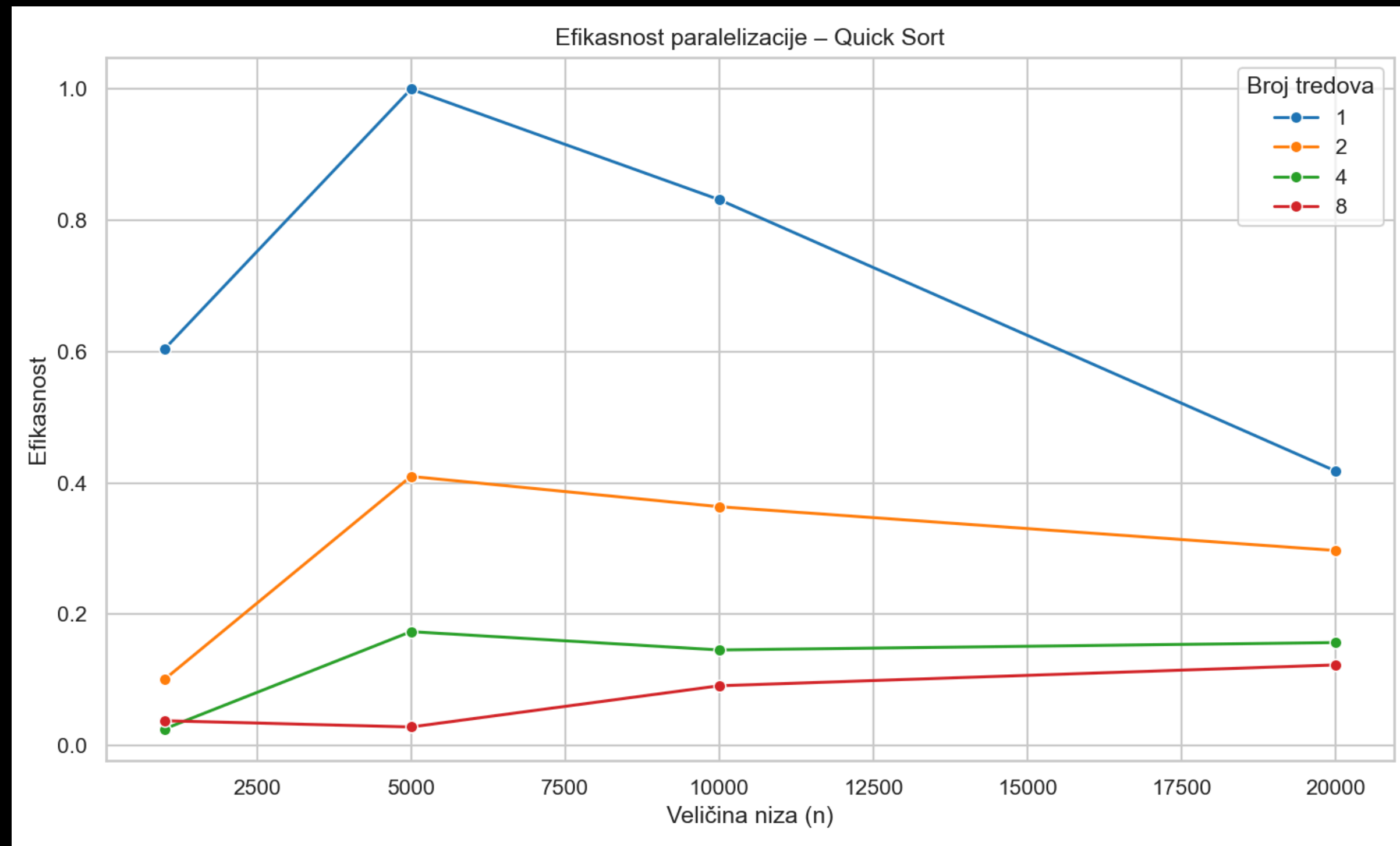
ANALIZA REZULTATA

- Najveći speedup (~1.5) ostvaren je sa 1 thread-om, što je u suštini sekvencijalni slučaj.
- Kod 2 i 4 thread-a speedup se kreće ispod 1 za sve veličine niza.
- Verzija sa 8 thread-ova pokazuje porast speedupa kod većih nizova ($n = 20000$).
- Nijedna paralelna konfiguracija ne ostvaruje linearan niti značajan speedup.

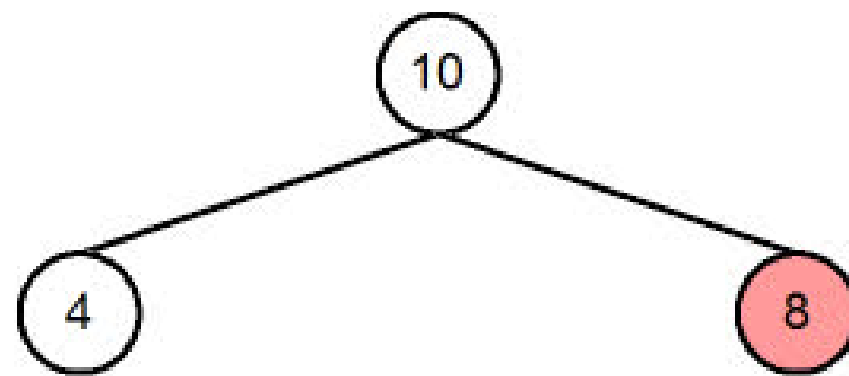
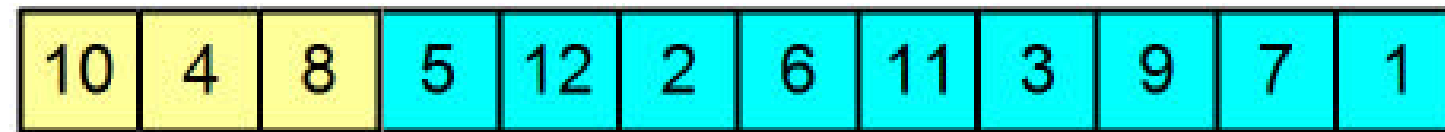


ANALIZA REZULTATA

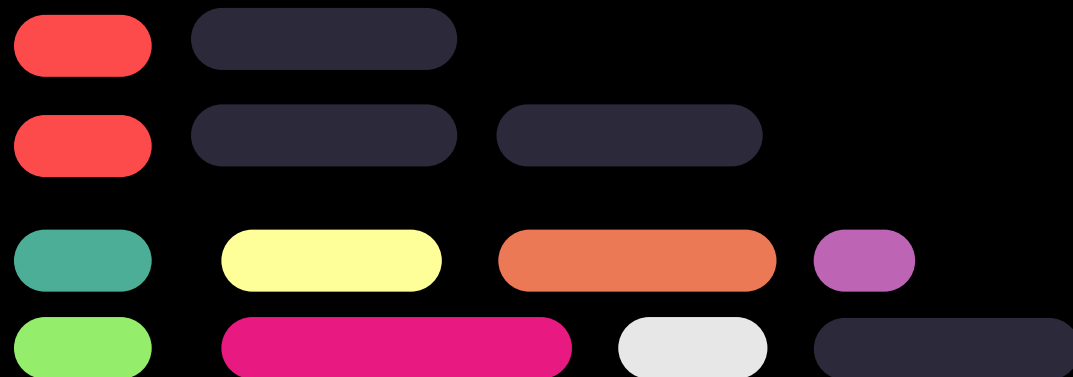
- Efikasnost značajno opada sa povećanjem broja thread-ova.
- Maksimalna efikasnost od 1.0 ostvarena je samo kod 1 thread-a.
- Verzije sa 4 i 8 thread-ova imaju konstantno nisku efikasnost (ispod 0.2).
- Čak ni kod većih nizova efikasnost ne pokazuje značajan rast — što ukazuje na ograničenu skalabilnost.

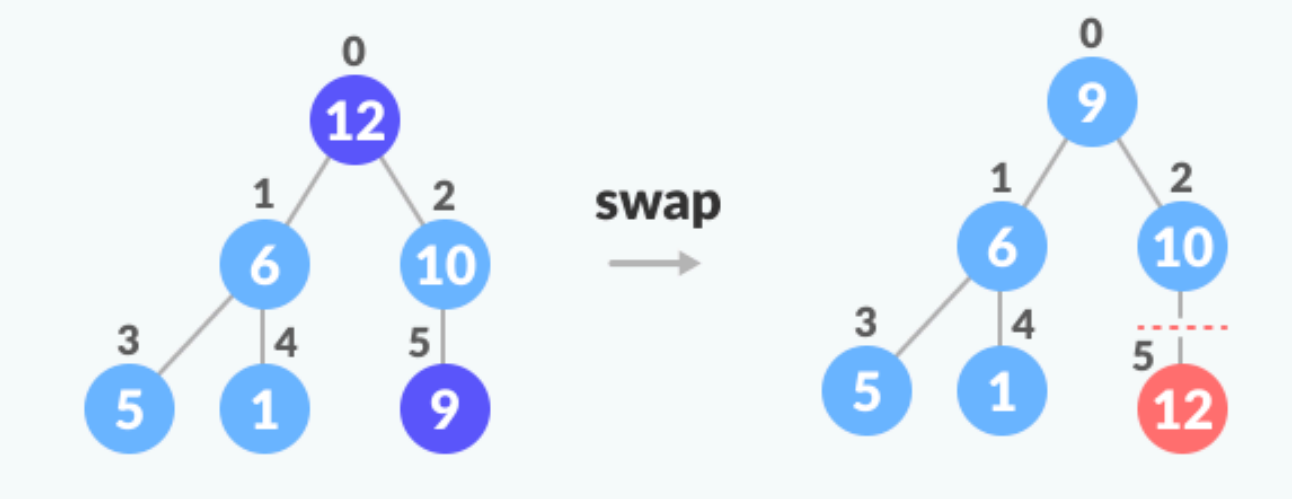


HEAP SORT

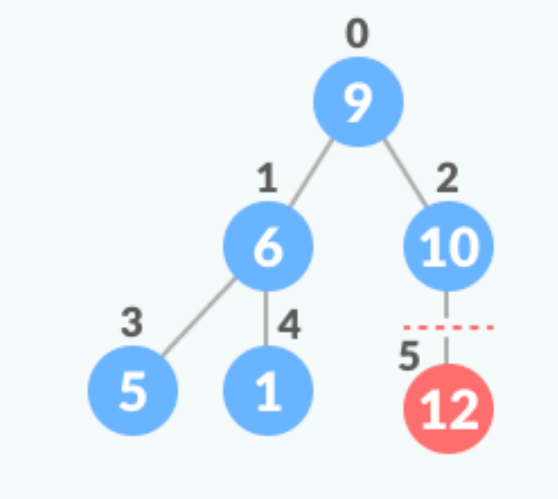


- Heap sort koristi maks-heap – potpuno binarno stablo gdje je roditelj veći ili jednak od potomaka.
- Može se efikasno predstaviti u običnom nizu koristeći matematički odnos između indeksa (lijevo dijete: $2i+1$, desno dijete: $2i+2$, roditelj: $\lfloor (i-1)/2 \rfloor$).
- Algoritam: Heap sort prvo pretvara niz u maks-heap, zatim uklanja najveći element i prilagođava preostali niz u heap.
- Pogodan za rad sa velikim količinama podataka u sistemima s ograničenim resursima.

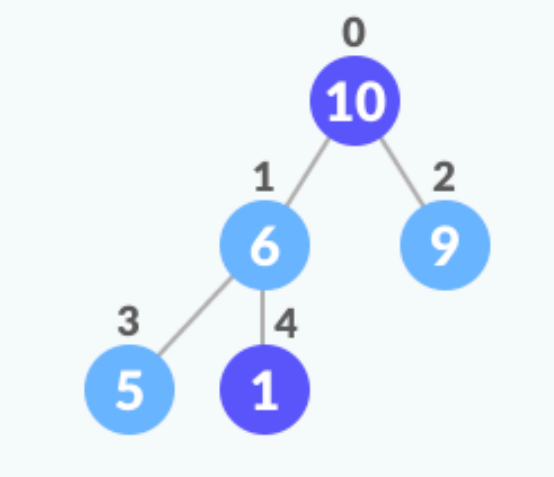




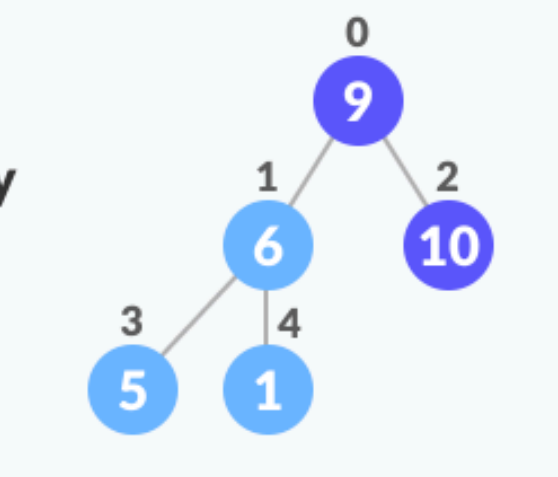
swap



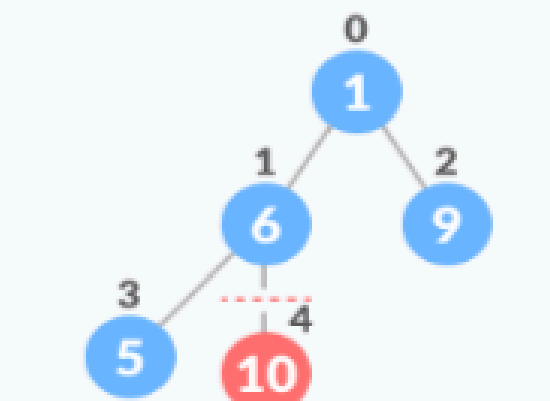
remove



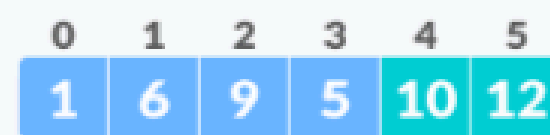
heapify



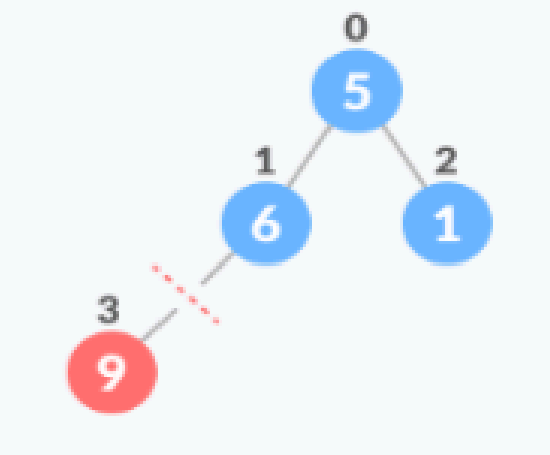
swap



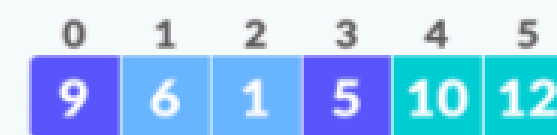
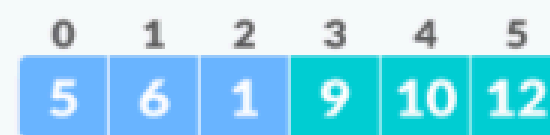
remove



heapify



swap



- *Swap (zamjena): Najveći element (na vrhu) se zamjenjuje sa posljednjim elementom u nizu. Na taj način najveći element dolazi na svoje konačno mjesto u sortiranom nizu, na samom kraju.*
- *Remove (uklanjanje): Nakon zamjene, veličina heap-a se smanjuje za 1, jer je najveći element već sortirani i više nije dio heap-a.*
- *Heapify: Korijenski element (koji je zamijenjen manjim) možda više ne zadovoljava heap svojstvo, pa se poziva funkcija heapify da se obnovi Max-heap struktura u preostalom dijelu niza.*

Ova tri koraka se ponavljaju sve dok svi elementi ne budu sortirani – tj. dok heap ne postane prazan.

Sekvencijalni algoritam

```
void heapify(int* arr, int n, int i) {
    int largest = i;
    int l = 2 * i + 1;
    int r = 2 * i + 2;

    if (l < n && arr[l] > arr[largest])
        largest = l;
    if (r < n && arr[r] > arr[largest])
        largest = r;

    if (largest != i) {
        std::swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}

void heapSort(int* arr, int n) {
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for (int i = n - 1; i >= 0; i--) {
        std::swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}
```

Paralelni algoritam

```
void heapifyParallel(int* arr, int n, int i) {
    int largest = i;
    int l = 2 * i + 1;
    int r = 2 * i + 2;

    if (l < n && arr[l] > arr[largest])
        largest = l;
    if (r < n && arr[r] > arr[largest])
        largest = r;

    if (largest != i) {
        std::swap(arr[i], arr[largest]);
        heapifyParallel(arr, n, largest);
    }
}

void heapSortParallel(int* arr, int n) {
    #pragma omp parallel for
    for (int i = n / 2 - 1; i >= 0; i--)
        heapifyParallel(arr, n, i);

    for (int i = n - 1; i >= 0; i--) {
        std::swap(arr[0], arr[i]);
        heapifyParallel(arr, i, 0);
    }
}
```

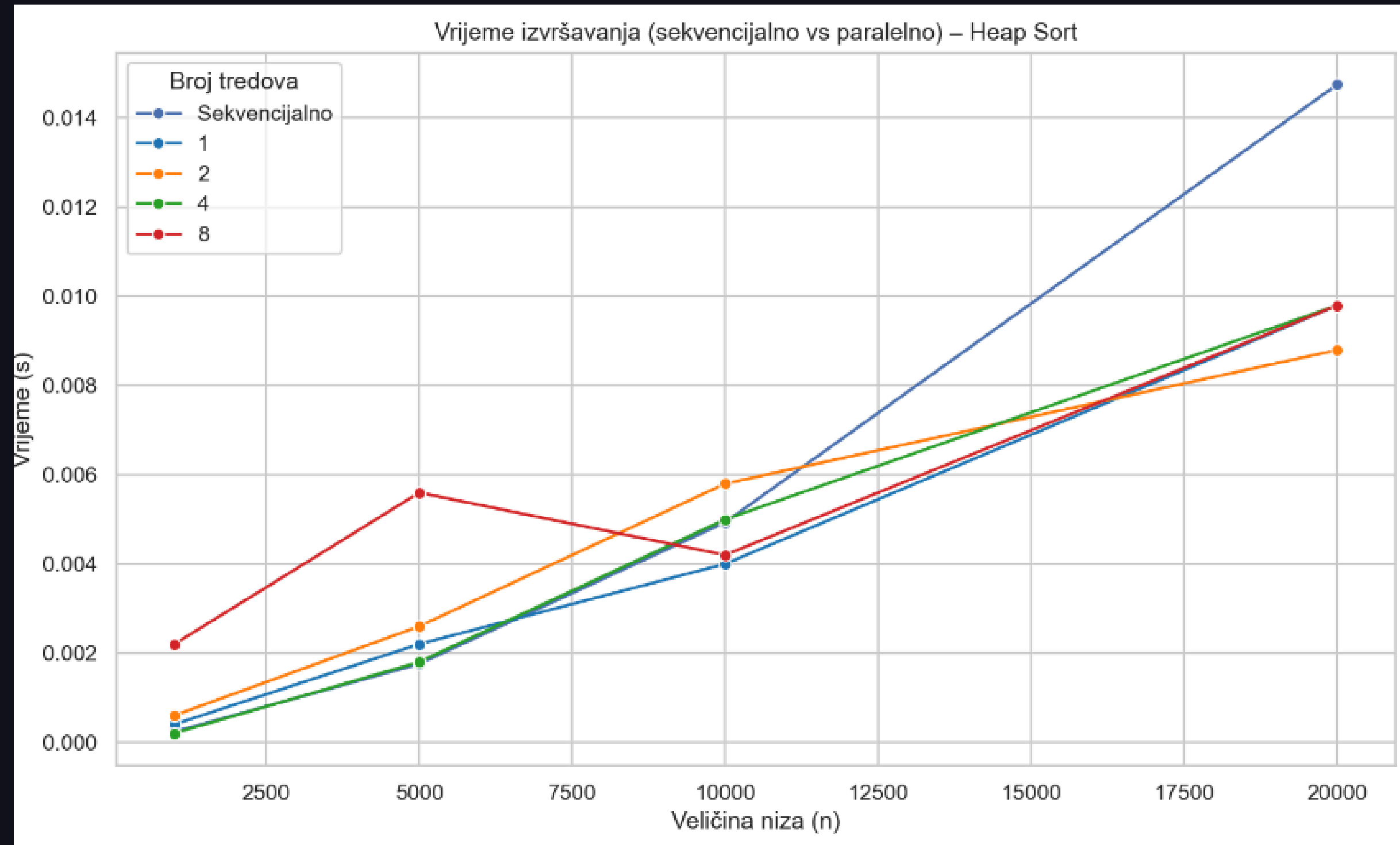
ANALIZA REZULTATA

n	threads	seq_time	par_time	speedup	efficiency	cost
1000.0	1.0	0.000234	0.0004	0.584723	0.584723	0.0004
1000.0	2.0	0.000234	0.0006	0.389831	0.194915	0.0012
1000.0	4.0	0.000234	0.0002	1.169585	0.292396	0.0008
1000.0	8.0	0.000234	0.0022	0.106319	0.01329	0.0176
5000.0	1.0	0.001757	0.0022	0.79877	0.79877	0.0022
5000.0	2.0	0.001757	0.0026	0.675876	0.337938	0.0052
5000.0	4.0	0.001757	0.0018	0.97626	0.244065	0.0072
5000.0	8.0	0.001757	0.0056	0.313801	0.039225	0.0448
10000.0	1.0	0.004934	0.004	1.233536	1.0	0.004
10000.0	2.0	0.004934	0.0058	0.850713	0.425356	0.0116
10000.0	4.0	0.004934	0.005	0.986824	0.246706	0.02
10000.0	8.0	0.004934	0.0042	1.174787	0.146848	0.0336
20000.0	1.0	0.014755	0.0098	1.50558	1.0	0.0098
20000.0	2.0	0.014755	0.0088	1.676674	0.838337	0.0176
20000.0	4.0	0.014755	0.0098	1.505573	0.376393	0.0392
20000.0	8.0	0.014755	0.0098	1.505565	0.188196	0.0784

- Najveći speedup (1.676) postignut je za $n = 20000$ sa 2 niti, ali efikasnost ostaje ispod 1.
- Kod malih nizova ($n = 1000$ i $n = 5000$), paralelizacija je uglavnom neefikasna – efikasnost često ispod 0.2.
- Za veće nizove ($n = 10000$ i $n = 20000$), vidljivo je blago poboljšanje, ali uz rast troška (cost).
- Efikasnost brzo opada sa povećanjem broja niti – npr. za 8 niti i $n = 20000$, efikasnost je samo 0.188.

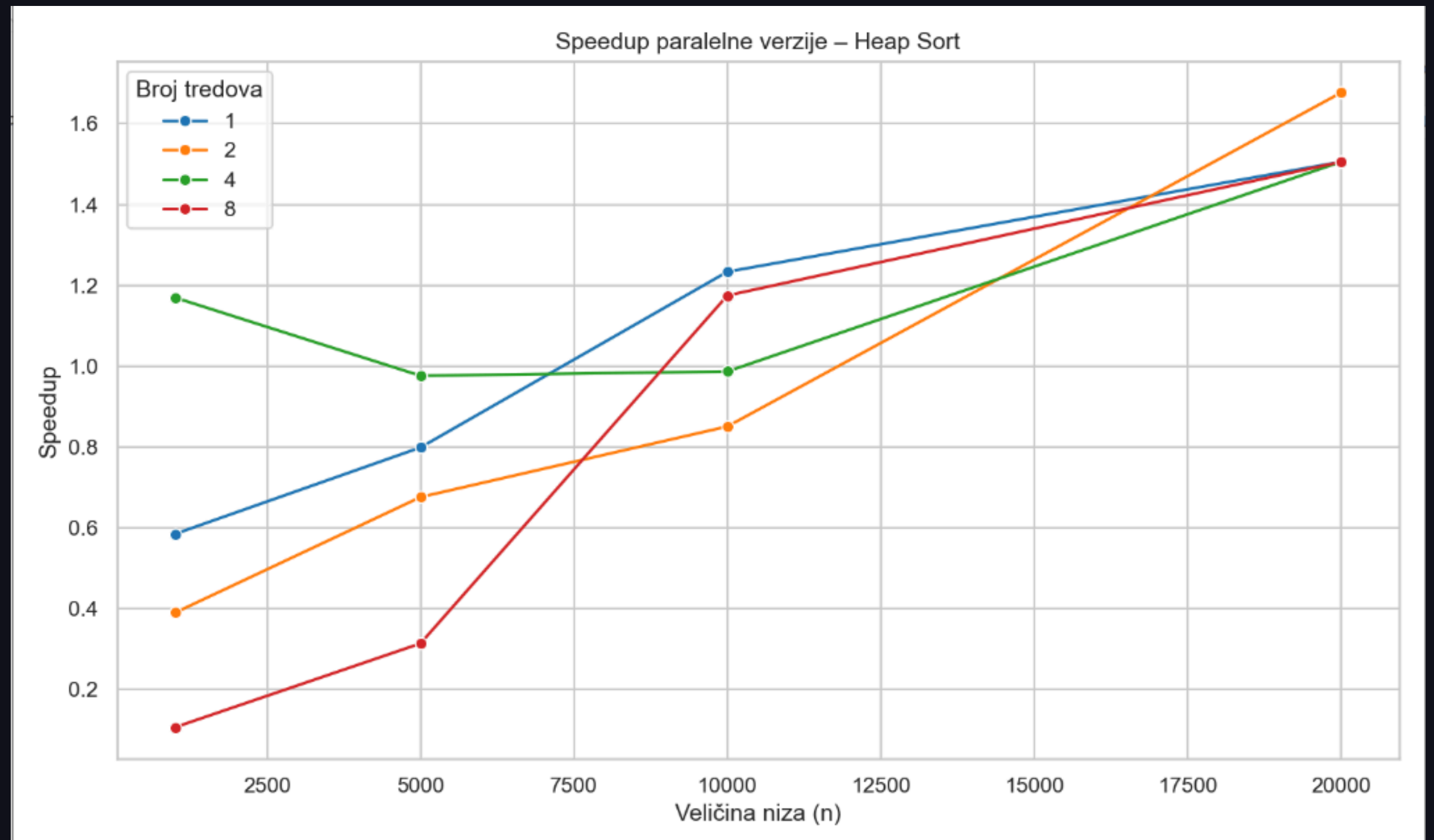
ANALIZA REZULTATA

- Vrijeme izvršavanja sekvencijalne verzije značajno raste sa veličinom niza, što je očekivano.
- Paralelne verzije (posebno sa 2 i 4 niti) postižu bolje vrijeme samo kod većih ulaza ($n = 20000$).
- Verzija sa 8 niti pokazuje oscilacije i kod srednjih veličina niza ($n = 5000$) ima veće vrijeme od ostalih.
- Paralelizacija donosi blago ubrzanje, ali samo pri većim ulazima – kod manjih nizova trošak nadmašuje dobitak.



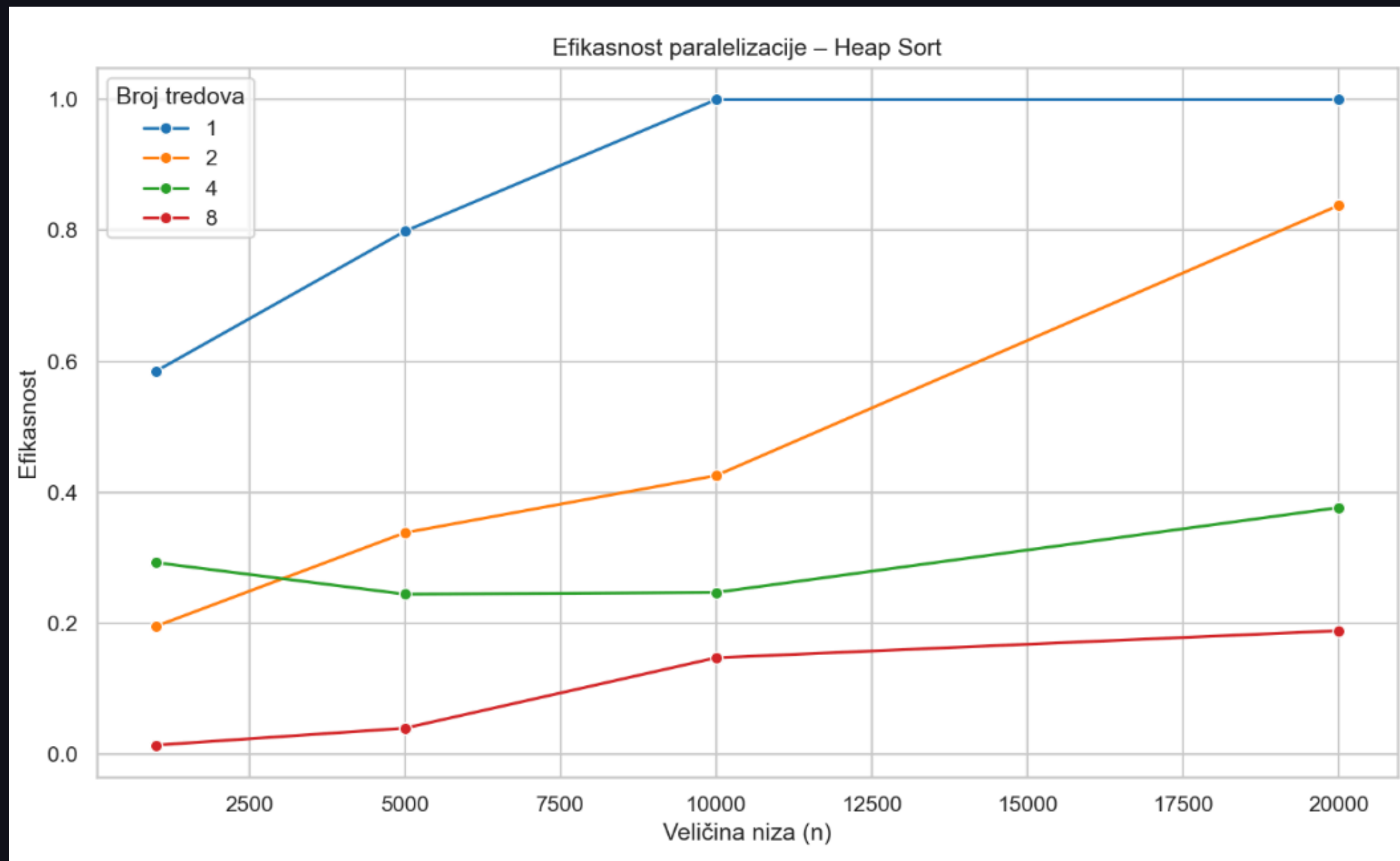
ANALIZA REZULTATA

- Speedup raste sa veličinom niza za sve konfiguracije.
- Verzija sa 8 niti dostiže najveći speedup (1.55) za $n = 20000$.
- Za male ulaze ($n = 1000\text{--}5000$), speedup je nizak i varira, naročito kod 8 niti.
- Paralelizacija ima smisla tek kod većih nizova, gdje se kompenzuje overhead.



ANALIZA REZULTATA

- Najveća efikasnost (1.0) ostvarena je očekivano sa 1 thread-om.
- Efikasnost kod 2 niti raste sa veličinom niza i prelazi 0.8 za $n = 20000$.
- Verzije sa 4 i 8 niti imaju znatno nižu efikasnost, posebno za manje nizove.
- Više niti = niža efikasnost, jer se trošak upravljanja tredovima ne isplati na malim ulazima.



ZAŠTO PARALELIZACIJA NIJE USPJEŠNA KOD HEAP SORT-A:

- Glavni problem je sekvencijalna priroda sortiranja – svaki korak zavisi od prethodnog (heapify nakon svakog vađenja korijena).
- Paralelizacija je moguća samo u buildHeap fazi, koja je vremenski vrlo kratka u odnosu na cijeli algoritam.
- Dodatni tredovi uvode overhead bez značajne koristi, što je posebno izraženo kod malih nizova.
- Zbog toga Heap Sort ima veoma ograničenu skalabilnost, što potvrđuju i eksperimentalni rezultati.

HvaĽa na paŹnĵi !